VIZSGAREMEK

BOLYAI 2025

Széplaki Dominik

Virág Benedek

VIZSGAREMEK – BOLYAI 2025

Budapesti Műszaki Szakképzési Centrum

Bolyai János Műszaki Technikum és Kollégium

FELADATLAP

SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ SZAKMA

vizsgaremek készítéséhez

A fejlesztő csapat tagjai: Széplaki Dominik és Virág Benedek

A dolgozat címe: GuideMate

A belső konzulens neve: Dorogházi Donát

A feladat leírása:

A feladat egy olyan webalkalmazás létrehozása, amely segíti a felhasználókat az utazásaik tervezésében és nyomon követésében. A felhasználók rögzíthetnek fontos eseményeket, például repülőjegy-foglalásokat és szállásfoglalásokat, valamint az alkalmazás segítségével könnyedén átláthatják az utazásuk részleteit. Indulás előtti check-listát tudnak létrehozni, hogy semmi ne maradjon otthon, és biztosak legyenek abban, hogy mindent elintéztek az utazás előtt. A cél, hogy a felhasználók stresszmentes és élvezetes utazási élményben részesüljenek.

A dolgozatnak tartalmaznia kell:

- Bevezetés
- Specifikáció
- Fejlesztői dokumentáció
- Tesztelői dokumentáció
- Felhasználói dokumentáció
- Összegzés

Beadási határidő: 2026. április 24.

Kiadva: 2025. szeptember 26.

..... Király László igazgató

A dolgozatot beadásra alkalmasnak tartom:

..... belső konzulens

kelt: Budapest, 2025.09.26

TANULÓI NYILATKOZAT

Alulírott tanuló kijelentem, hogy a vizsgaremek saját munkám eredménye, a felhasznált szakirodalmat és eszközöket azonosíthatóan közöltem. Egyéb jelentős segítséget nem vettem igénybe. Az elkészült szakdolgozatban található eredményeket a technikum saját céljaira térítés nélkül felhasználhatja.

Kelt: ……

..... tanuló aláírása

Tartalomjegyzék

1. Bevezetés ... 5
1.1 Az alkalmazásról ... 6
1.2 A megoldandó probléma ... 6
1.3 A projekt célja ... 7
2. Specifikáció ... 7
2.1 A program rövid leírása ... 8
2.2 Felhasznált technológiák ... 8
2.3 Jogosultságok és szerepkörök ... 9
2.4 Követelmények ... 10
2.4.1 Funkcionális követelmények ... 10
2.4.2 Nem funkcionális követelmények ... 10
2.5 Célok és célkitűzések ... 11
2.6 Alkalmazási területek ... 11
2.7 Fejlesztési módszertan ... 11
2.8 A csapat tagjai és szerepköreik ... 12
3. Fejlesztői dokumentáció ... 12
3.1 Tervezői rész ... 13
3.1.1 Rendszerarchitektúra ... 13
3.1.2 Adatbázis-terv ... 13
3.1.3 API-tervezés ... 17
3.1.4 Frontend architektúra ... 19
3.2 Megvalósítás ... 20
3.2.1 Hitelesítési modul ... 20
3.2.2 Utazáskezelő modul ... 20
3.2.3 Napi program modul és dátumkiterjesztés ... 21
3.2.4 Geolokáció és képkeresés modul ... 21
3.2.5 Adminisztrációs modul ... 21
3.2.6 Frontend megvalósítás ... 22
3.2.7 Adatbázis és ORM ... 22
3.3 Fejlesztői rendszerkövetelmények és telepítés ... 23
4. Tesztelői dokumentáció ... 23
4.1 Tesztelési stratégia ... 24
4.2 Unit tesztelés ... 24
4.2.1 Backend unit tesztek ... 24
4.2.2 Frontend komponenstesztek ... 25

4.3 Funkcionális tesztelés ... 25
4.4 Rendszerteszt ... 26
4.5 Blackbox tesztelés ... 26
5. Felhasználói dokumentáció ... 27
5.1 Rendszerkövetelmények ... 28
5.2 Az alkalmazás használata ... 28
5.2.1 Regisztráció ... 28
5.2.2 Bejelentkezés ... 28
5.2.3 Utazások kezelése ... 29
5.2.4 Napi programok kezelése ... 30
5.2.5 Interaktív térkép ... 31
5.2.6 Jegykövetés (foglalások kezelése) ... 33
5.2.7 Ellenőrző listák ... 34
5.2.8 Profilkezelés ... 36
5.2.9 Adminisztrációs felület ... 37
6. Összefoglalás ... 38
7. Kitekintés ... 39
7.1 Lehetséges továbbfejlesztési irányok ... 40
7.1.1 Kollaboratív funkciók ... 40
7.1.2 Mobilalkalmazás ... 40
7.1.3 AI-alapú ajánlórendszer ... 40
7.1.4 Időjárás-integráció ... 40
7.1.5 Exportálás és megosztás ... 40
7.1.6 Valutaváltó és költségkövetés ... 41

1. Bevezetés

A GuideMate egy webes utazástervező alkalmazás, amit azért hoztunk létre, hogy az embereknek ne kelljen egyszerre több különböző oldalt és appot használniuk, amikor nyaralást terveznek. Az ötlet egyszerű: legyen egy hely, ahol mindent el lehet intézni – a tervezéstől egészen az utazás végéig.

1.1 Az alkalmazásról

Az alkalmazás két fő részből áll: egy React-alapú webes felületből, amit a felhasználók látnak és használnak, illetve egy NestJS-ben írt szerverből, ami a háttérben kezeli az adatokat. Az adatokat MariaDB adatbázisban tároljuk, és a Prisma-val eszközzel kommunikálunk vele, ami megkönnyíti és biztonságosabbá teszi az adatkezelést.

Ahol szükséges, külső szolgáltatásokat is beépítettünk: helyszínek és látnivalók kereséséhez a Nominatim (OpenStreetMap) és a Geoapify API-t, képekhez a Pexels API-t, ország- és fővárosadatokhoz pedig a REST Countries API-t használjuk.

Az app összesen hat főbb dolgot tud: utazásokat lehet benne tervezni és kezelni, napi programokat összerakni, repülő-, busz-, vonat-, autó- és szállásfoglalásokat nyilvántartani, interaktív térképen látnivalókat keresni, személyre szabható ellenőrző listát készíteni, illetve az adminisztrátoroknak saját felületet biztosít a rendszer kezeléséhez.

1.2 A megoldandó probléma

Ha valaki ma el akar utazni valahová, általában rengeteg különböző appot és oldalt kell egyszerre nyitva tartania. Van egy külön alkalmazás a repülőjegyekhez, egy másik a szálláshoz, egy naptár a programokhoz, és még Google Maps is nyitva van a látnivalók miatt. Ez az egész nagyon gyorsan átláthatatlanná válik, és könnyen előfordul, hogy az ember elfelejtett valamit lefoglalni, vagy rosszul ütemezte a programokat.

Csoportos utazásnál ez még problémásabb. Mindenki más forrásból tájékozódik, nehéz összehangolni, ki mit vállalt, és ki miről értesült. Nincs egy közös felület, ahol

a csoport minden tagja egyszerre látná az aktuális menetrendet, a foglalásokat és a közös teendőket.

1.3 A projekt célja

A GuideMate célja, hogy ezt az egész széttöredezett folyamatot egy helyre hozza össze. A fejlesztés során a következő fő dolgokra fókuszáltunk:

- Komplex utazáskezelés: több utazást is egyszerre lehet kezelni, és minden utazáshoz külön tartalmakat – programokat, foglalásokat, teendőlistákat – lehet hozzárendelni.
- Napi programtervezés: naptárnézetben lehessen napirendet összerakni, és látnivalókat hozzárendelni az egyes napokhoz.
- Foglalások nyomon követése: repülő-, busz-, vonat-, autó- és szállásfoglalási adatok tárolása, áttekinthető formában megjelenítve.
- Interaktív térképintegráció: helyszíneket és látnivalókat lehessen keresni és szűrni valós idejű térképen.
- Teendőlisták kezelése: az utazáshoz kapcsolódó listákat létre lehessen hozni, és nyomon lehessen követni, mi van már kész.
- Biztonságos bejelentkezés: JWT-alapú hitelesítéssel, ahol az adminisztrátoroknak és a sima felhasználóknak külön jogosultságaik vannak.

2. Specifikáció

2.1 A program rövid leírása

A GuideMate egy webes alkalmazás, amelynek célja, hogy támogassa a felhasználókat utazásaik teljes folyamatában: az ötlettől és inspirációtól kezdve a részletes programtervezésen át a konkrét jegy- és szállásfoglalások nyilvántartásáig.

A rendszer egyik fő funkciója az interaktív térkép, amely lehetővé teszi, hogy a felhasználó városokra keresve megtekintse azok legfontosabb látnivalóit. Például ha a felhasználó Budapestre keres rá, azonnal láthatja a főbb turisztikai helyeket, mint a Budai Vár, az Országház vagy a Hősök tere. Ezeket a látnivalókat egy kattintással hozzáadhatja az utazási tervéhez.

A rendszer fő célja egy könnyen kezelhető, felhasználóbarát és reszponzív alkalmazás létrehozása, amelyben a felhasználók egy helyen kezelhetik az utazással kapcsolatos összes információjukat.

A rendszer fő moduljai:

- Utazástervező: utazások létrehozása, dátumok és helyszínek megadása, napi programok összeállítása.
- Térképes látnivalók: a térképen megjelenő látnivalók keresése és hozzáadása az utazási ütemtervhez.
- Foglaláskövetés: repülőjegyek, buszjegyek, vonatjegyek, autóbérlés és szállásfoglalások adatainak rögzítése és nyilvántartása.
- Ellenőrző lista: személyre szabható lista, amely segít, hogy semmi fontos ne maradjon otthon.

2.2 Felhasznált technológiák

Frontend

- TypeScript – típusbiztos JavaScript felépítmény
- React – felhasználói felület keretrendszere
- Vite – gyors fejlesztői környezet és csomagolóeszköz
- Chakra UI – stílus- és komponenskeretrendszer

- Leaflet és React Leaflet – interaktív térképmegjelenítés
- Mapbox – térkép csempék (tile réteg) megjelenítéséhez (streets, satellite, outdoors, light, dark stílusok)

Backend

- TypeScript – típusbiztos szerveroldali fejlesztés
- NestJS – moduláris, REST API keretrendszer
- Prisma ORM – adatbázis-kezelő réteg típusbiztos lekérdezésekkel
- Passport.js és JWT – hitelesítés és jogosultságkezelés

Adatbázis

Az alkalmazás adatait MariaDB 11.4 relációs adatbázis kezeli, amely Docker konténerben fut.

Tesztelés és verziókezelés

- Verziókezelés: GitHub
- Backend tesztelés: Jest keretrendszer
- Frontend tesztelés: Vitest és React Testing Library
- Feladatkövetés: Trello
- API tesztelés: Postman

Fejlesztői környezet

- Visual Studio Code

2.3 Jogosultságok és szerepkörök

Vendég

- Láthatja az interaktív térképet
- Megtekintheti a látnivalókat
- Böngészheti a helyszíneket

Regisztrált felhasználó

- Utazásokat hozhat létre és kezelhet
- Látnivalókat adhat hozzá az utazási tervhez

- Mentheti foglalásait (repülő, busz, vonat, autó, szállás)
- Használhatja az ellenőrző lista funkciót
- Módosíthatja profilját és jelszavát

Adminisztrátor

- Megtekintheti a rendszer statisztikáit
- Kezeli a felhasználói fiókokat (keresés, szerepkör módosítás, törlés)
- Hozzáfér az összes adminisztrációs funkcióhoz

2.4 Követelmények

2.4.1 Funkcionális követelmények

1. Interaktív térkép: Helyszínre keresve jelenjenek meg a főbb látnivalók, amelyek hozzáadhatók az utazási tervhez.
2. Utazások létrehozása: A felhasználó megadhatja az utazás nevét, dátumtartományát és leírását.
3. Programtervezés: A felhasználó napokra bontva adhat hozzá programokat időpontokkal.
4. Foglalások rögzítése: A felhasználó rögzítheti repülőjegy-, vonat-, autóbérlés- vagy szállásfoglalás adatait.
5. Ellenőrző lista: Személyre szabható lista, amelynek elemei kipipálhatók és törölhetők.
6. Bejelentkezés és regisztráció: E-mail és jelszó alapú felhasználói fiók létrehozása és bejelentkezés.

2.4.2 Nem funkcionális követelmények

- Teljesítmény: Az alkalmazás gyors és zökkenőmentes felhasználói élményt biztosít, minimális betöltési idővel.
- Reszponzivitás: A felület minden eszközön megfelelően jelenik meg (mobil, tablet, asztali számítógép).
- Felhasználói élmény: Az alkalmazás felépítése logikus és átlátható navigációt biztosít.

- Biztonság: A felhasználói jelszavak titkosítva (hash-elve) kerülnek tárolásra, a hitelesítés JWT tokennel történik.
- Karbantarthatóság: A rendszer moduláris felépítése lehetővé teszi a könnyű módosítást és bővítést.

2.5 Célok és célkitűzések

A projekt célja egy olyan webalkalmazás létrehozása, amely:

- Vizsgán bemutatható, működőképes fő funkciókkal rendelkezik.
- Segíti a felhasználókat az utazások megtervezésében, rendszerezésében és követésében.
- Csökkenti az utazás előtti stresszt az ellenőrzőlisták segítségével.
- Egy helyen tárolja az utazással kapcsolatos összes információt.
- Modern, intuitív és reszponzív felhasználói felületet biztosít.
- A jövőben továbbfejleszthető, akár valós felhasználásra is alkalmas rendszer alapját képezheti.

2.6 Alkalmazási területek

- Egyéni utazások: Például egy párizsi hosszú hétvége megszervezése.
- Családi nyaralás: Közös programok tervezése, éttermek és szállások rögzítése.
- Osztálykirándulás: A tanár összeállíthatja a diákok programját, az ellenőrző lista segítségével.

2.7 Fejlesztési módszertan

A projekt fejlesztése iteratív módszertan szerint történik.

Iteratív fejlesztés

- Heti sprintekben történő fejlesztés, rendszeres egyeztetésekkel.

Kanban módszer

A feladatokat Trello táblán követjük nyomon, három oszlopba rendezve: To Do (teendő), In Progress (folyamatban), Done (kész).

Kódfelülvizsgálat

- Minden feladat merge előtt code review-n megy keresztül GitHub-on.

Tesztelés

- API tesztek Postman segítségével
- Frontend és backend automatikus tesztek
- Manuális funkcionális tesztelés böngészőből

2.8 A csapat tagjai és szerepköreik

Széplaki Dominik

- Backend fejlesztés (NestJS, Prisma, adatbázis)
- API tervezés és megvalósítás
- Tesztelés

Virág Benedek

- Frontend fejlesztés (React, Chakra UI)
- UI/UX tervezés és megvalósítás
- Tesztelés

3. Fejlesztői dokumentáció

A fejlesztői dokumentáció két nagyobb részre osztható: az egyik a tervezési döntéseket mutatja be – tehát az architektúrát, az adatbázis felépítését és az API tervezési elveket –, a másik pedig azt írja le, hogyan valósítottuk meg az egyes modulokat a gyakorlatban.

3.1 Tervezői rész

3.1.1 Rendszerarchitektúra

Az alkalmazás egy klasszikus háromrétegű architektúrára épül. Ez azt jelenti, hogy a három fő rész – a frontend, a backend és az adatbázis – egymástól elkülönítve fejleszthető és tesztelhető, és jól meghatározott felületeken keresztül kommunikálnak egymással.

- Prezentációs réteg (Frontend): Ez az a rész, amit a felhasználó lát. Egy React 18 alapú egyoldalas webalkalmazás, amit Vite fejlesztőszerver futtat az 5173-as porton. A felület Chakra UI komponensekből épül fel, az oldalak közötti navigációt pedig React Router DOM v7 kezeli.
- Üzleti logika réteg (Backend): Ez a háttérben futó szerver, ami a 3000-es porton érhető el, és NestJS 11 keretrendszerre épül. Az egyes funkciók – mint az utazások, foglalások vagy programok – külön NestJS modulokban vannak szervezve, mindegyiknek saját vezérlője, szolgáltatása és adatátviteli objektumai (DTO) vannak.
- Adatréteg (Adatbázis): Az adatokat egy MariaDB 11.4 relációs adatbázisban tároljuk, ami Docker konténerben fut a 3306-os porton. A backend Prisma ORM 7-en keresztül kommunikál vele, ami típusbiztos és automatikusan generált klienst biztosít.
- Külső API-k: Néhány funkcióhoz külső szolgáltatásokat is igénybe veszünk: a Nominatim geocodinghoz (helynév–koordináta átalakítás), a Geoapify helykereséshez és látnivalókhoz, a REST Countries fővárosadatokhoz, a Pexels pedig képkereséshez. Ezeket a backenden a GeoService és az ImagesService kezeli.

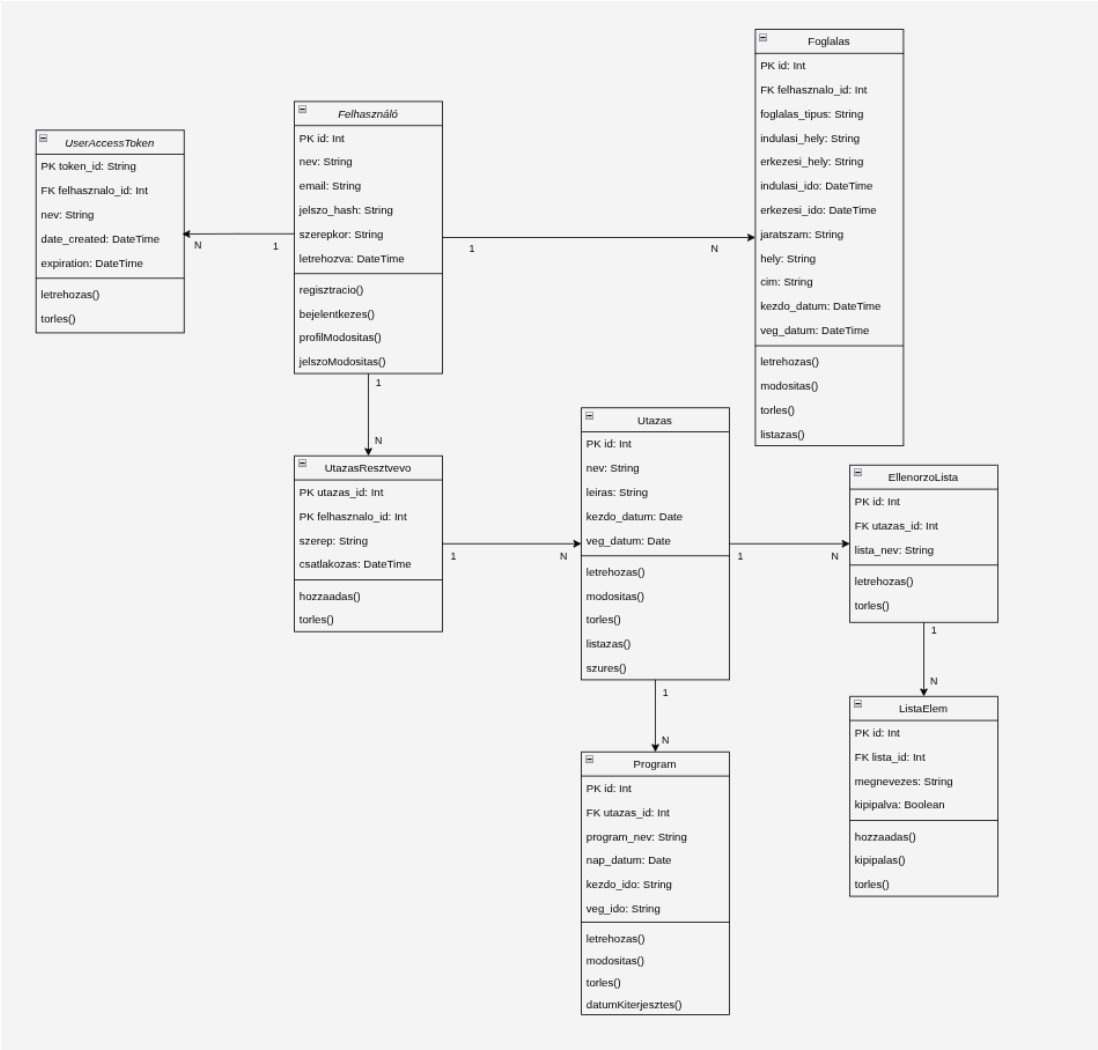
3.1.2 Adatbázis-terv

Az adatbázis sémáját Prisma schema nyelven írtuk meg, és MariaDB alatt fut. A séma összesen nyolc fő entitást tartalmaz, amelyek az alkalmazás fő adatobjektumait képviselik.

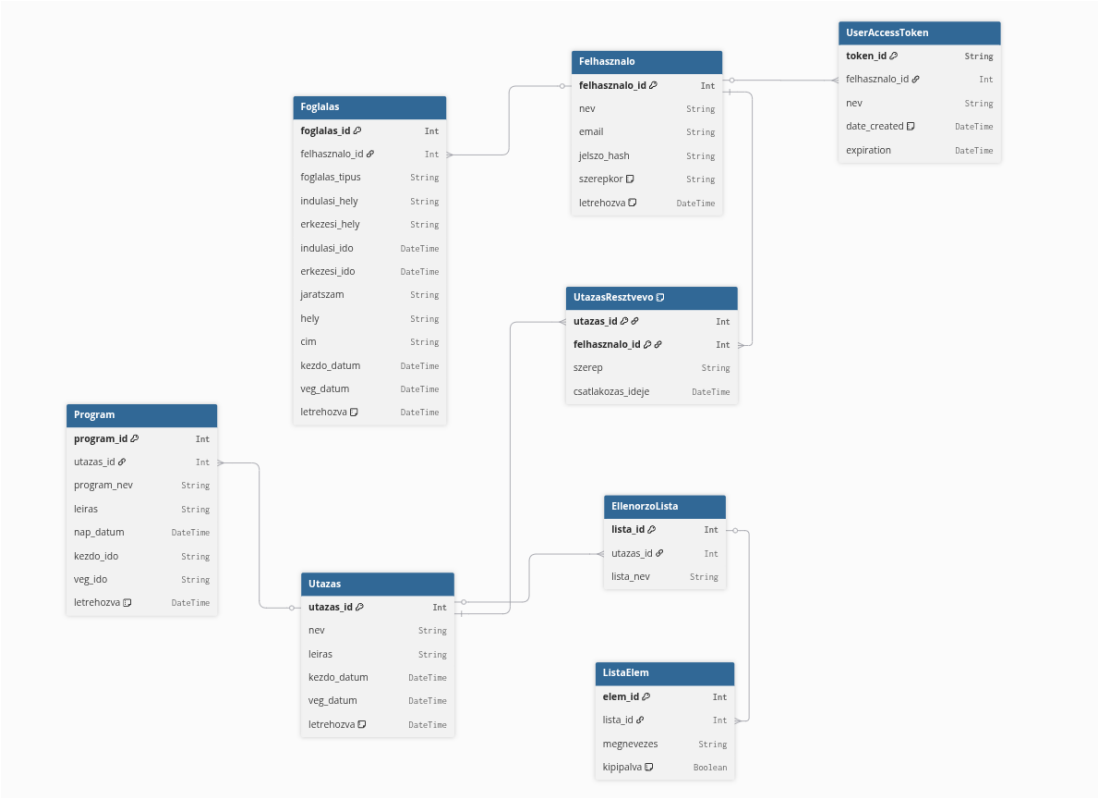
A Felhasznalo entitás tárolja a regisztrált felhasználókat – nevet, e-mail címet, jelszó hasht és szerepkört. Az Utazas entitás egy-egy utazást jelöl, névvel, leírással, kezdő és záró dátummal. A kettő között egy UtazasResztvevo kapcsolótábla áll, amelynek összetett elsődleges kulcsa van (utazás azonosító + felhasználó azonosító), és további mezőket is tartalmaz, mint a szerep és a csatlakozás időpontja. Ez teszi lehetővé, hogy egy utazásnak több résztvevője legyen, és egy felhasználó több utazásban is szerepeljen.

A Program entitás egy adott nap programját tárolja dátummal, kezdő és záró időponttal. A Foglalas entitás közvetlenül a felhasználóhoz kapcsolódik egy-a-többhöz kapcsolattal, és egy típusmező határozza meg, hogy repülő-, busz-, vonat-, autó-, szállásfoglalásról van-e szó. Az EllenorzoLista az utazáshoz kapcsolódik, és ListaElem entitásokból áll, amelyek egy megnevezést és egy kipipálva/nem kipipálva állapotot tárolnak.

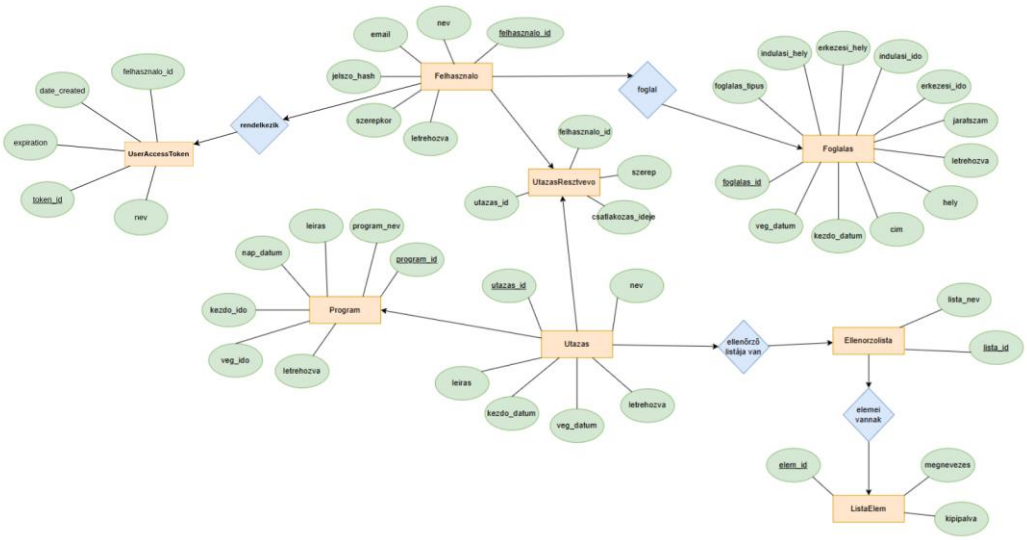
A helyszínek és látnivalók nem kerülnek be az adatbázisba – ezeket valós időben kérjük le a Geoapify és Nominatim API-któl, és közvetlenül a térképes felületen jelenítjük meg.



1. ábra: UML osztálydiagram az entitások mezőivel és metódusaival



2. ábra: Az adatbázis fizikai diagramja (ER diagram)



3. ábra: Az adatbázis egyed-relációs diagramja

1. táblázat: Az adatbázis entitásainak összefoglalása

Entitás	Fő mezők	Kapcsolatok
Felhasznalo	id, nev, email, jelszo_hash, szerepkor	Utazas (N:M), Foglalas (1:N), UserAccessToken (1:N)
Utazas	id, nev, leiras, kezdo_datum, veg_datum	Felhasznalo (N:M), Program, EllenorzoLista

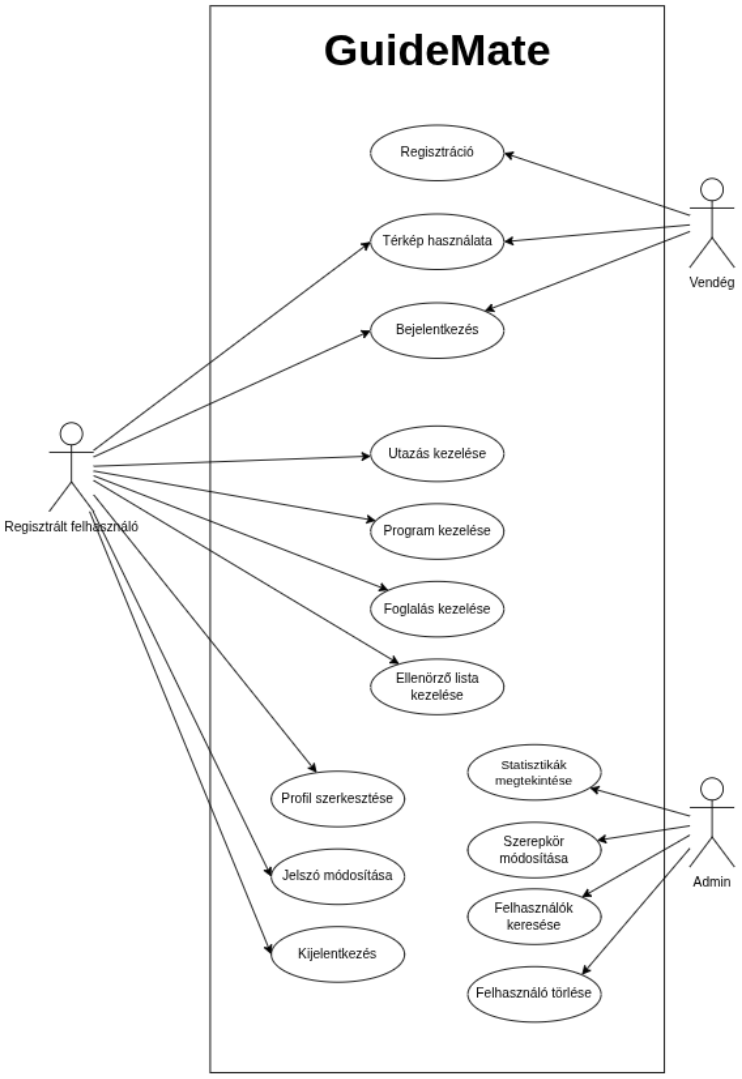
UtazasResztvevo	utazas_id, felhasznalo_id, szerep, csatlakozas_ideje	Utazas + Felhasznalo (kapcsolótábla)
Program	id, utazas_id, program_nev, leiras, nap_datum, kezdo_ido, veg_ido	Utazas (N:1)
Foglalas	id, felhasznalo_id, foglalas_tipus, indulasi_hely, erkezesi_hely, indulasi_ido, erkezesi_ido, jaratszam, hely, cim, kezdo_datum, veg_datum	Felhasznalo (N:1)
EllenorzoLista	id, utazas_id, lista_nev	Utazas (N:1), ListaElem (1:N)
ListaElem	id, lista_id, megnevezes, kipipalva	EllenorzoLista (N:1)
UserAccessToken	token_id, felhasznalo_id, nev, date_created, expiration	Felhasznalo (N:1)

3.1.3 API-tervezés

A backend API-ja RESTful elvek alapján működik. A különböző műveletek a szabványos HTTP metódusokhoz vannak rendelve: GET-tel lehet adatot lekérni, POST-tal újat létrehozni, PUT-tal meglévőt frissíteni, DELETE-tel pedig törölni.

A válaszokban egységes, magyar mezőneveket használunk, például azonosító, cím, leírás, kezdő dátum, sikeres vagy üzenet. Ennek köszönhetően az adatbázis belső felépítése és az API által visszaadott formátum egymástól függetlenül változtatható – ha az egyiken módosítunk, nem feltétlenül kell a másikat is átírni.

A bejelentkezés és a védett végpontok JWT (JSON Web Token) alapú hitelesítéssel védettek. Sikeres bejelentkezés után a kliens kap egy tokent, amit ezután minden olyan kérésnél el kell küldenie az Authorization fejlécben, amelyhez be kell jelentkezni. A token tartalmazza a felhasználó azonosítóját és e-mail címét, így a szerver mindig tudja, ki küldte a kérést.



4. ábra: Use-case diagram a rendszer főbb használati eseteiről

2. táblázat: A backend főbb API-végpontcsoportjai

Végpontcsoport	**Metódus + útvonal**	**Funkció**
Hitelesítés	POST /api/auth/bejelentkezes	Bejelentkezés, JWT generálás
	POST /api/auth/regisztracio	Regisztráció
Felhasználó	GET /api/felhasznalok/profil	Saját profil lekérése
	PUT /api/felhasznalok/profil	Név módosítása
	PUT /api/felhasznalok/jelszo	Jelszó módosítása
Utazások	GET /api/utazasok	Utazások listázása
	POST /api/utazasok	Új utazás létrehozása
	GET /api/utazasok/:id	Utazás részletei
	PUT /api/utazasok/:id	Utazás frissítése

	DELETE /api/utazasok/:id	Utazás törlése (kaszkád)
Programok	GET /api/utazasok/:utazasId/programok	Programok listázása
	POST /api/utazasok/:utazasId/programok	Program létrehozása
	PUT /api/programok/:id	Program frissítése
	DELETE /api/programok/:id	Program törlése
Foglalások	GET /api/foglalasok	Foglalások listázása
	POST /api/foglalasok	Foglalás létrehozása
	PUT /api/foglalasok/:id	Foglalás frissítése
	DELETE /api/foglalasok/:id	Foglalás törlése
Ellenőrző listák	GET /api/utazasok/:utazasId/ellenorzolista	Listák lekérdezése
	POST /api/utazasok/:utazasId/ellenorzolista	Lista létrehozása
	DELETE /api/ellenorzolista/:id	Lista törlése
Listaelemek	POST /api/ellenorzolista/:listaId/elemek	Elem hozzáadása
	PUT /api/ellenorzolista/elemek/:id	Elem frissítése (kipipálás)
	DELETE /api/ellenorzolista/elemek/:id	Elem törlése
Geolokáció	GET /api/geo/coordinates	Helyszín geocoding
	GET /api/geo/places	Látnivalók keresése
	GET /api/geo/search	Helyszín szöveges keresése
Képek	GET /api/images	Képkeresés (Pexels, gyorsítótárazott)
Admin	GET /api/admin/felhasznalok	Felhasználók listázása
	GET /api/admin/felhasznalok/:id	Felhasználó részletei
	PUT /api/admin/felhasznalok/:id	Szerepkör módosítása
	DELETE /api/admin/felhasznalok/:id	Felhasználó törlése
	GET /api/admin/statisztikak	Rendszerstatisztikák

3.1.4 Frontend architektúra

A frontend egy egyoldalas alkalmazás, amelynek mappaszerkezete funkcionálisan elkülönített részekre tagolódik:

- oldalak/: Itt találhatók az egyes oldalakat megvalósító React komponensek. Minden oldalnak saját almappája van (például UtazastervezoOldal/, TerkepOldal/ vagy JegyKovetesOldal/), és az adott oldalhoz tartozó

TypeScript típusdefiníciók is itt kapnak helyet az oldal mellett (például utazastervezo.types.ts).

- features/: Ez az API-kommunikációs réteg. Minden funkcióterületnek – utazás, foglalás, hitelesítés, program, ellenőrző lista, admin stb. – saját almappája van, ahol az API-hívó függvények vannak, elkülönítve a felületi logikától.
- app/: Az alkalmazás alapvető konfigurációját tartalmazza: az útvonalkezelést (router), az elrendezést (layout) és a kontextus-szolgáltatókat (providers).
- lib/: Segédfüggvények és konfigurációk, például az API-kliens beállítása és a térképes szolgáltatásokhoz (OpenTripMap) kapcsolódó segédeszközök.
- komponensek/: Újrafelhasználható UI elemek.
- test/: A teszteléshez szükséges konfigurációs és segédfájlok (setup, utils).

3.2 Megvalósítás

3.2.1 Hitelesítési modul

A bejelentkezést és a regisztrációt az AuthService kezeli. Regisztrációnál először megnézi, hogy az adott e-mail cím már foglalt-e, majd BCrypt algoritmussal hash-eli a jelszót 10 salt-körrel, és létrehozza a felhasználói rekordot. Bejelentkezésnél a megadott jelszót összehasonlítja a tárolt hash-sel, és ha egyeznek, JWT tokent állít ki, ami tartalmazza a felhasználó azonosítóját és e-mail címét.

A JwtStrategy (passport-jwt) minden védett kérésnél automatikusan lefut: kicsomagolja a tokent, és a benne lévő azonosító alapján betölti a felhasználói rekordot. Az eredmény a kérés user mezőjébe kerül, ahonnan a vezérlők kinyerik a bejelentkezett felhasználó adatait.

3.2.2 Utazáskezelő modul

Az UtazasService minden adatbázis-műveletet a PrismaService-en keresztül végez. A listázásnál egyetlen Prisma-tranzakcióban fut le a darabszám-lekérdezés és az adatlekérdezés, a szűrési, rendezési és lapozási feltételekkel együtt. Minden lista- és részlet-végpont ellenőrzi, hogy a kérő felhasználó valóban résztvevője-e az adott utazásnak – ha nem, 403-as hibával utasítja el a kérést.

Törléskor a szolgáltatás szintén tranzakciót használ, hogy a kapcsolódó rekordok a helyes sorrendben törlődjenek: először a listaelemek (ListaElem), majd a programok (Program), az ellenőrző listák (EllenorzoLista) és a résztvevői rekordok (UtazasResztvevo), végül maga az utazás (Utazas). Ez gondoskodik arról, hogy az adatbázisban ne maradjanak árva hivatkozások.

3.2.3 Napi program modul és dátumkiterjesztés

A ProgramService figyeli, hogy az újonnan hozzáadott program dátuma belefér-e az utazás jelenlegi időkeretébe, és ha nem, automatikusan kiterjeszti a kezdő vagy záró dátumot. Ennek köszönhetően a felhasználónak nem kell kézzel módosítgatnia az utazás dátumait – elég csak felvenni az új programot, a többi magától igazodik.

3.2.4 Geolokáció és képkeresés modul

A GeoService három külső API-t kezel egyszerre. A helyszínkereséshez a Nominatim API-t hívja (helynév alapján koordinátákat keres), a látnivalókereséshez a Geoapify Places API-t (koordináták alapján közeli érdekességeket), a főváros-adatokhoz pedig a REST Countries API-t. A Nominatim használati szabályzata szerint minden híváshoz megfelelő User-Agent fejléc is el van küldve.

A térkép csemperétegét (tile layer) a Mapbox API biztosítja, amelyet a Leaflet jelenít meg. A Mapbox több térképstílust is támogat: streets, satellite, outdoors, light és dark.

A képkeresést az ImagesService végzi, ami memóriaalapú gyorsítótárral (Map) tárolja a Pexels API válaszait. A cache legfeljebb 500 bejegyzést tárol, és minden elem 6 óra után érvényét veszti. Így kevesebb külső API-hívás szükséges, a képek pedig gyorsabban töltődnek be.

3.2.5 Adminisztrációs modul

Az AdminService minden funkciója először megvizsgálja, hogy a kérő felhasználónak van-e adminisztrátori jogosultsága. Ha nincs, a rendszer 403-as hibával utasítja el a kérést – ez egy egységes biztonsági minta, amit az összes admin funkció következetesen alkalmaz.

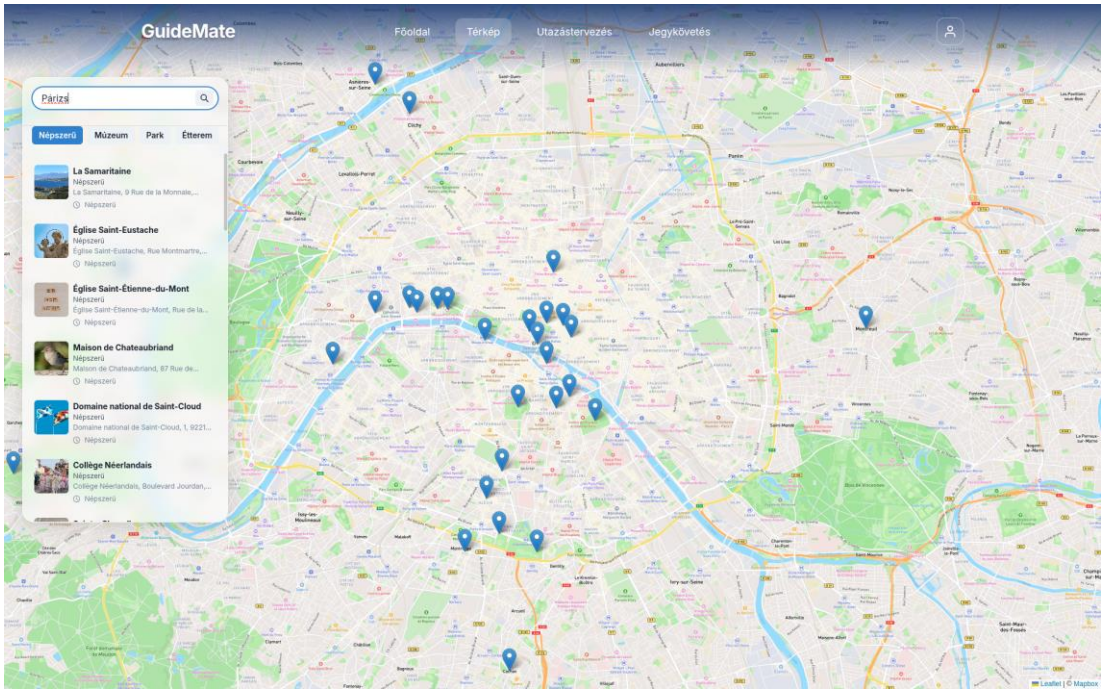
Felhasználó törlésekor a rendszer egyetlen tranzakcióban törli a felhasználó hozzáférési tokenjeit (UserAccessToken), foglalásait és résztvevői rekordjait, végül magát a felhasználói fiókot.

A statisztikai végpont egyetlen tranzakcióban 11 Prisma-lekérdezést futtat, és visszaadja a felhasználó-, utazás-, program-, foglalás- és listaszámokat, valamint a napi aktivitási adatokat is, például a mai regisztrációkat és újonnan létrehozott utazásokat.

3.2.6 Frontend megvalósítás

Az útvonalkezelést React Router DOM v7 végzi. A védett oldalakat egy AuthGuard komponens védi, ami megnézi, hogy megvan-e a bejelentkezési token (gm_token) a localStorage-ban – ha nincs, visszairányítja a felhasználót a bejelentkezési oldalra.

Az interaktív térkép Leaflet és React Leaflet alapú. A TerkepOldal komponens kezeli a helyszínkeresést (Nominatim), a látnivalókeresést (Geoapify) és a jelölők megjelenítését. A találatok kategóriánként szűrhetők: népszerű helyek, múzeumok, parkok, éttermek. A képkeresés a Pexels API-n keresztül, gyorsítótárazott válaszokkal működik.



5. ábra: A TerkepOldal felülete helyszínkereséssel és Leaflet jelölőkkel

Az utazástervező főoldal (UtazastervezoOldal) tartalmazza a saját utazások listáját kártyanézetben, egy oldalsávban pedig az ellenőrző listák kezelhetők. Az UtReszletekOldal az utazás részleteit és a napi programok idővonalát jeleníti meg. Az ellenőrzőlista panel bal oldalt helyezkedik el, és egy legördülő menüt tartalmaz az utazások közötti váltáshoz. Az "Általános lista" opció egy utazástól független, localStorage-ban tárolt listát jelenít meg; utazás kiválasztása esetén a rendszer az adott utazáshoz tartozó ellenőrzőlistát tölti be a szerverről.

6. ábra: Az utazástervező főoldalának felülete

3.2.7 Adatbázis és ORM

A Prisma séma a prisma/schema.prisma fájlban van meghatározva, és itt szerepelnek az összes modell és kapcsolat definíciói. A Prisma generátor előállítja a @prisma/client csomagot, ami típusbiztos lekérdezési felületet ad a szolgáltatásoknak. Ha a sémán változtatni kell, a Prisma Migrate automatikusan legenerálja és alkalmazza a szükséges SQL migrációs fájlokat.

A Docker Compose egy MariaDB 11.4 konténert indít a 3306-os porton. Az adatok egy nevesített Docker kötetben (db_data) tárolódnak, így a konténer újraindítása után sem vesznek el. A konténer 5 másodpercenként ellenőrzi saját elérhetőségét egy beépített healthcheck segítségével.

3.3 Fejlesztői rendszerkövetelmények és telepítés

Az alkalmazás futtatásához az alábbi szoftverek szükségesek:

- Node.js v20 LTS vagy újabb
- Docker és Docker Compose
- Git verziókezelő

Telepítési lépések:

1. lépés – Forráskód letöltése: git clone https://github.com/[repository-url]/GuideMate.git cd GuideMate

2. lépés – Backend beállítása: cd backend cp .env.example .env A .env fájlban töltse ki: DATABASE_URL, JWT_SECRET, PEXELS_API_KEY, GEOAPIFY_API_KEY

3. lépés – Adatbázis indítása (Docker): docker compose up -d npx prisma migrate deploy

4. lépés – Backend indítása: npm install npm run start:dev

5. lépés – Frontend indítása: cd ../frontend/GuideMate npm install npm run dev

Az alkalmazás ezt követően a http://localhost:5173 címen érhető el. A backend API a http://localhost:3000 porton hallgat.

4. Tesztelői dokumentáció

Ez a fejezet ismerteti az alkalmazáshoz kapcsolódó tesztelési tevékenységeket: az alkalmazott tesztelési stratégiát, a backend és frontend unit tesztjeit, a manuális funkcionális teszteket, a rendszerszintű tesztelést és a blackbox teszteseteket.

4.1 Tesztelési stratégia

A GuideMate projektben kétféle automatizált tesztelési szintet alkalmazunk. A backend unit tesztjei Jest 30 keretrendszerrel, TypeScript támogatással (ts-jest) futnak, és az egyes szolgáltatás-osztályok metódusait tesztelik elkülönítetten, a Prisma és egyéb függőségek helyettesítésével (mock-olásával). A frontend komponenstesztjei Vitest és React Testing Library segítségével készültek, a felhasználói műveletek szimulálásával.

A backend Jest konfigurációja 80%-os minimális lefedettséget ír elő az elágazások, függvények, sorok és utasítások tekintetében egyaránt.

A lefedettségből kivett modulok: prisma.service, geo, images, latnivalo, helyszin (ezek külső API-függőségük vagy infrastrukturális jellegük miatt nem alkalmasak unit tesztelésre).

4.2 Unit tesztelés

4.2.1 Backend unit tesztek

A backend nyolc szolgáltatás-osztályt tesztel unit szinten. Minden tesztfájl a NestJS Testing Module segítségével elkülönített tesztkörnyezetet hoz létre, ahol a Prisma-függőségeket és a külső szolgáltatásokat helyettesítő (mock) objektumokkal pótolja. A tesztek a sikeres és sikertelen eseteket egyaránt lefedik: sikeres végrehajtás, nem talált erőforrás (NotFoundException), jogosultság hiánya (ForbiddenException) és érvénytelen adatok (BadRequestException).

3. táblázat: Backend unit tesztek összefoglalása

Service	Tesztelt esetek	Főbb tesztelt funkciók
AuthService	4	Bejelentkezés (siker, rossz jelszó), Regisztráció (siker, foglalt email)

UsersService	7	Keresés ID/email alapján, létrehozás, profil- és jelszófrissítés, utazásszám
UtazasService	11	Listázás (szűrés, lapozás, rendezés), részletek, létrehozás, frissítés, törlés
ProgramService	10	Listázás, létrehozás (dátumkiterjesztés), frissítés, törlés, jogosultság
FoglalasService	8	Listázás, létrehozás, frissítés (nem talált, tiltott, siker), törlés
EllenorzoListaService	5	Listázás (tiltott, siker), létrehozás, törlés (nem talált, siker)
ListaElemService	10	Létrehozás (nem talált, tiltott, siker), frissítés, törlés
AdminService	11	Listázás (szűrés, keresés), részletek, szerepkör, törlés, statisztikák

4.2.2 Frontend komponenstesztek

A frontend öt tesztfájlt tartalmaz, amelyek a Vitest és React Testing Library együttes használatával tesztelik az oldalkomponenseket. Az API-hívások vi.mock() segítségével helyettesítve vannak, a felhasználói műveleteket a @testing-library/user-event szimulálja.

4. táblázat: Frontend komponenstesztek összefoglalása

Komponens / tesztfájl	Főbb tesztelt esetek
BejelentkezesOldal	Bejelentkezési API hívása megadott adatokkal, JWT tárolása a böngésző helyi tárában
RegisztracioOldal	Üres mezők ellenőrzése, jelszóegyezés vizsgálata, API hívás helyes adatokkal, szóközvágás
JegyKovetesOldal	Foglalások szűrése keresési kifejezéssel (típus, helyszín, járatszám)
UjJegyForm	Repülőszám mező megjelenítése típustól függően, autónál elrejtése
UtazastervezoOldal	Utazáslista megjelenítése, betöltési állapot kezelése, új utazás gomb

4.3 Funkcionális tesztelés

A funkcionális tesztelés manuálisan, böngészőből és Postman segítségével zajlott. Az alábbi táblázat a legfontosabb teszteseteket foglalja össze:

5. táblázat: Funkcionális tesztesetek

#	Funkció	Tesztelési lépések	Várt eredmény

FT-01	Regisztráció	1. Főoldal > Regisztráció 2. Adatok kitöltése 3. Regisztrálás gomb	Sikeres regisztráció, átirányítás bejelentkezési oldalra
FT-02	Bejelentkezés	1. Email + jelszó megadása 2. Belépés gomb	JWT token tárolása, átirányítás a tervező oldalra
FT-03	Utazás létrehozása	1. Utazástervező > Új utazás 2. Adatok megadása 3. Mentés	Utazás megjelenik a listában
FT-04	Program létrehozása	1. Utazás részletei > Új program 2. Adatok, dátum, idő megadása 3. Mentés	Program megjelenik az idővonalban
FT-05	Helyszínkeresés térképen	1. Térkép oldal > Keresőmező 2. Város neve > Enter	Térkép a helyszínre ugrik, jelölők megjelennek
FT-06	Foglalás rögzítése	1. Jegykövetés > Új jegy 2. Típus, adatok megadása 3. Mentés	Foglalás megjelenik a listában
FT-07	Ellenőrző lista	1. Utazás > Lista létrehozása 2. Elemek hozzáadása 3. Kipipálás	Elemek állapota megváltozik és megmarad
FT-08	Admin: szerepkör	1. Admin panel > Felhasználók 2. Szerepkör módosítás gomb	Szerepkör frissül, visszajelzés megjelenik

4.4 Rendszerteszt

A rendszerteszt a teljes alkalmazás végponttól végpontig terjedő (end-to-end) viselkedését ellenőrzi valós adatbázis-kapcsolattal és böngészős felülettel. Az alábbi forgatókönyvek kerültek tesztelésre:

- Teljes utazástervezési folyamat: Regisztrációtól az utazás létrehozásán, programok felvételén és foglalások rögzítésén át az ellenőrző lista kezeléséig és az utazás törléséig terjedő teljes folyamat.
- Kaszkád törlés: Utazás törlése után ellenőrzés, hogy a programok, listák és résztvevői rekordok valóban törlődtek-e az adatbázisból.
- Jogosultságellenőrzés: Más felhasználó tokenjével való hozzáférési kísérlet 403 Forbidden választ ad.
- Párhuzamos API-hívások: A frontend egyszerre indít több API-kérést; a válaszok következetesen érkeznek.
- Képkeresés gyorsítótár: Ugyanazon kulcsszóra ismételt keresés a gyorsítótárból szolgálódik ki (a Pexels API nem kap ismételt hívást 6 órán belül).

4.5 Blackbox tesztelés

A blackbox tesztelés a rendszer viselkedését kizárólag a bemeneti és kimeneti adatokra összpontosítva vizsgálja, a belső megvalósítás ismerete nélkül. A tesztelés Postman segítségével, az API végpontokat közvetlenül hívva zajlott.

6. táblázat: Blackbox tesztesetek

#	Tesztelt funkció	Bemeneti adatok	Várt kimenet	Eredmény
BT -01	Sikeres regisztráció	Érvényes név, email, jelszó	201 Created	SIKERES
BT -02	Foglalt email	Már regisztrált email	400 Bad Request	SIKERES
BT -03	Sikeres bejelentkezés	Érvényes email + jelszó	200 OK, JWT token	SIKERES
BT -04	Hibás jelszó	Érvényes email, rossz jelszó	401 Unauthorized	SIKERES
BT -05	Utazás létrehozása	Érvényes cím, dátumok	201 Created	SIKERES
BT -06	Más utazása	Más user utazás ID-ja	403 Forbidden	SIKERES
BT -07	Szűrés (aktív)	?statusz=aktiv	Csak aktív utazások	SIKERES
BT -08	Program létrehozása	Érvényes adatok, dátum	201 Created	SIKERES
BT -09	Dátumkiterjesztés	Program utazáson kívüli dátummal	201 Created, dátum frissül	SIKERES
BT -10	Foglalás (repülő)	Repülő típus, érvényes adatok	201 Created	SIKERES
BT -11	Elem kipipálása	elem_id, kipipalva: true	200 OK, frissített	SIKERES
BT -12	Utazás törlése	Utazás ID (kapcsolt adatokkal)	200 OK, kaszkád törlés	SIKERES
BT -13	Admin: lista	Admin JWT token	200 OK, lapozott lista	SIKERES
BT -14	Admin normál userrel	Normál user JWT	403 Forbidden	SIKERES
BT -15	Képkeresés cache	Azonos lekérdezés ismételve	200 OK, gyorsítótárból	SIKERES

5. Felhasználói dokumentáció

Ez a fejezet a GuideMate alkalmazás telepítéséhez szükséges lépéseket, valamint az összes felhasználói funkció részletes kezelési útmutatóját tartalmazza.

5.1 Rendszerkövetelmények

- Webböngésző: Google Chrome 120+ vagy Mozilla Firefox 120+ (ajánlott)
- Képernyőfelbontás: Minimum 1280×720 px, ajánlott 1920×1080 px
- Internetkapcsolat: Szükséges a külső API-k (térkép, képkeresés) eléréséhez
- JavaScript: Engedélyezve kell lennie a böngészőben

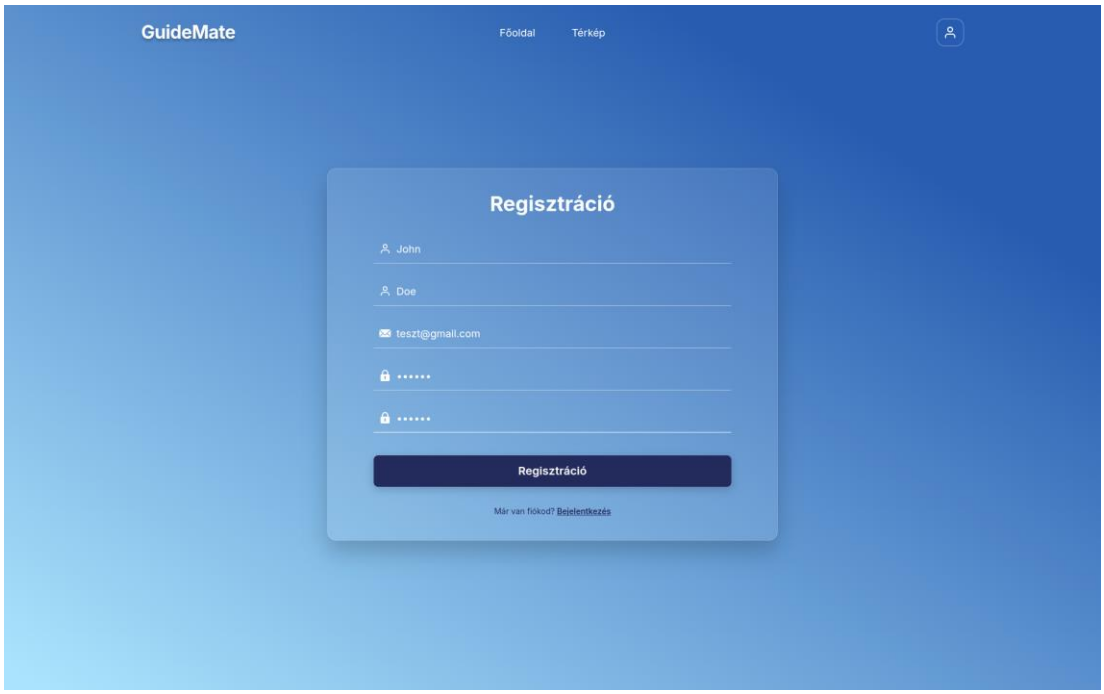
5.2 Az alkalmazás használata

5.2.1 Regisztráció

Új fiók létrehozásához kattintson a főoldalon a Regisztráció gombra. A megjelenő oldalon az alábbi mezőket kell kitölteni:

- Vezetéknév
- Keresztnév
- E-mail cím
- Jelszó
- Jelszó megerősítése

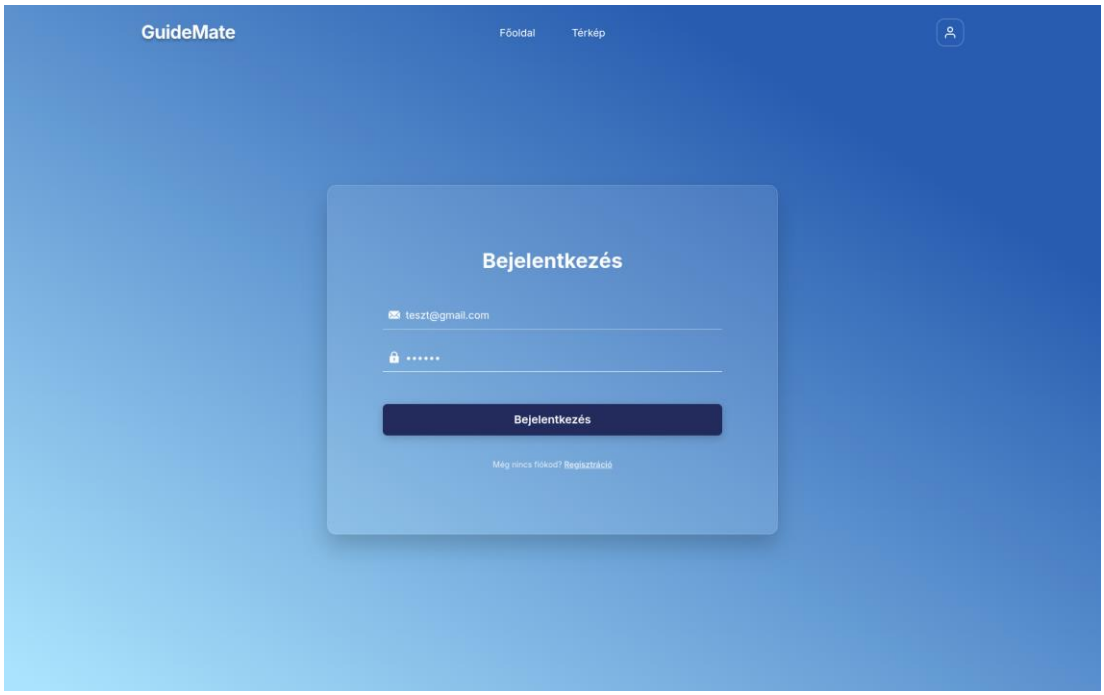
A mezők kitöltése után kattintson a Regisztrálás gombra. A rendszer ellenőrzi az adatok érvényességét: ha valamelyik mező üres, a jelszavak nem egyeznek, vagy az e-mail cím már foglalt, hibaüzenet jelenik meg. Sikeres regisztráció esetén a rendszer automatikusan átirányítja a bejelentkezési oldalra.



7. ábra: A regisztrációs oldal felülete

5.2.2 Bejelentkezés

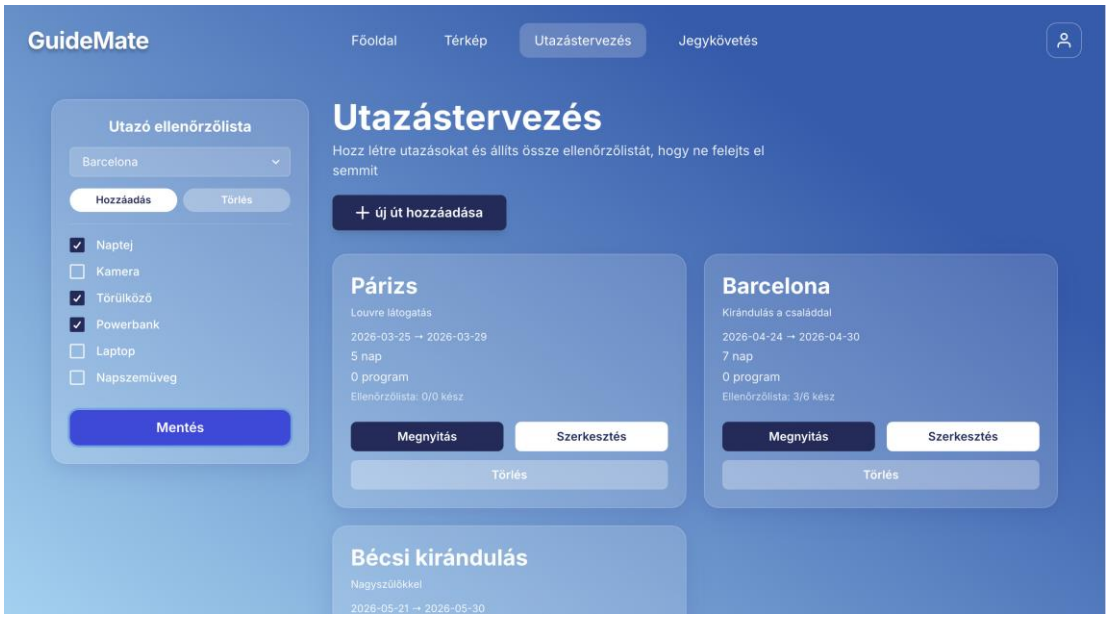
A bejelentkezési oldalon adja meg a regisztrált e-mail címét és jelszavát, majd kattintson a Belépés gombra. Sikeres bejelentkezés után a rendszer az Utazástervező oldalra irányítja. A munkamenet a böngésző helyi tárában (localStorage) tárolt JWT token segítségével marad fenn. Kijelentkezni a navigációs sáv jobb oldalán található felhasználói menü Kijelentkezés pontjával lehet.



8. ábra: A bejelentkezési oldal felülete

5.2.3 Utazások kezelése

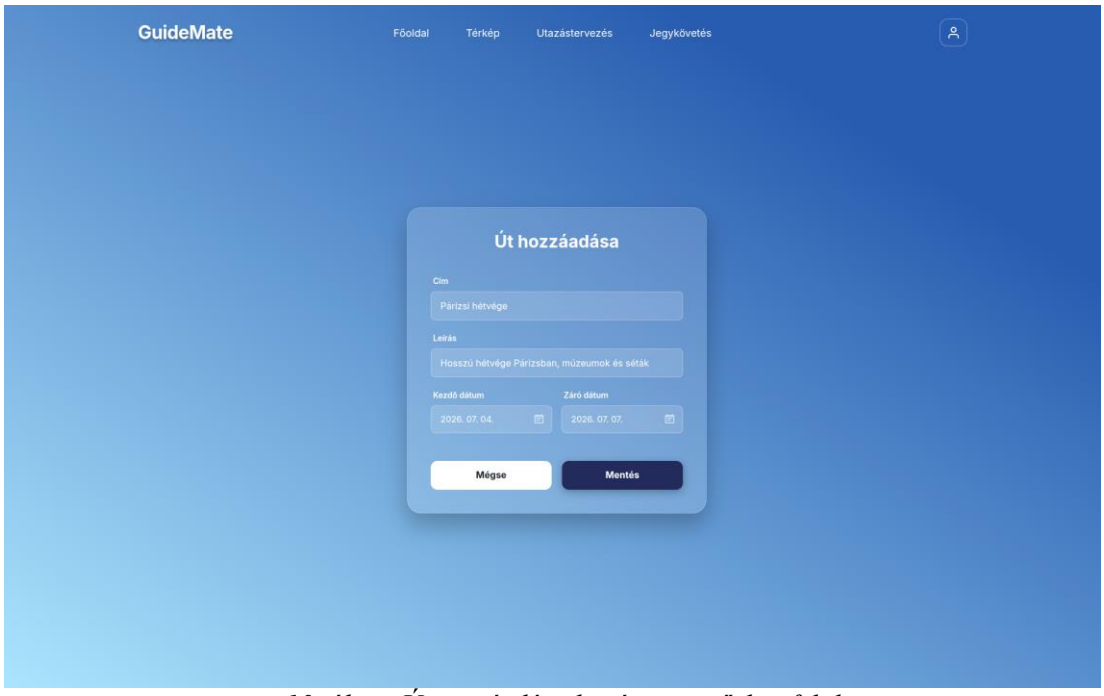
Az Utazástervező oldalon (főmenüpont) láthatja összes utazásának listáját. Minden utazás külön kártyán jelenik meg a nevével, dátumaival és állapotával.



9. ábra: Az utazástervező oldal utazáskártyákkal

Új utazás létrehozása

Az Utazástervező oldalon kattintson az „Új utazás” gombra. A megjelenő űrlapon adja meg az utazás nevét, rövid leírását, valamint a kezdő és záró dátumot. A Mentés gombra kattintva az utazás létrejön, és megjelenik a listában.



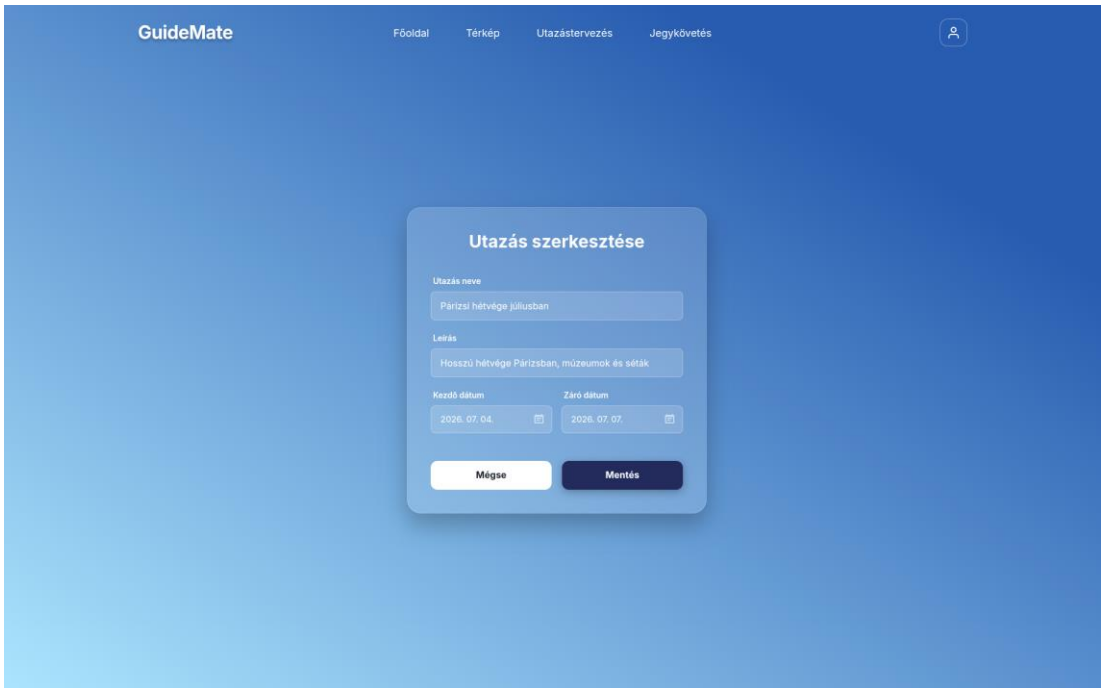
10. ábra: Új utazás létrehozása – az űrlap felülete

Utazás megtekintése

Az utazás nevére (kártyájára) kattintva megnyílik az utazás részletei oldal, ahol láthatja az utazás adatainak összefoglalóját, a napi programok idővonalát és az ellenőrző listákat.

Utazás szerkesztése

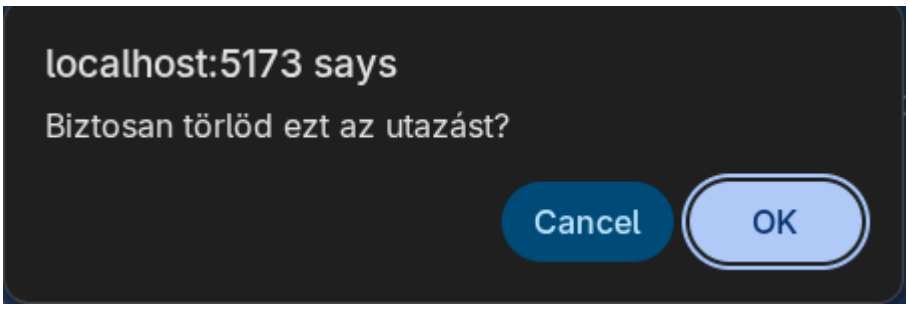
Az utazás részletei oldalon kattintson a szerkesztés ikonra (ceruza ikon). Módosíthatja az utazás nevét, leírását és dátumait. A változtatásokat a Mentés gombra kattintva véglegesítheti.



11. ábra: Utazás szerkesztése – a szerkesztő felület

Utazás törlése

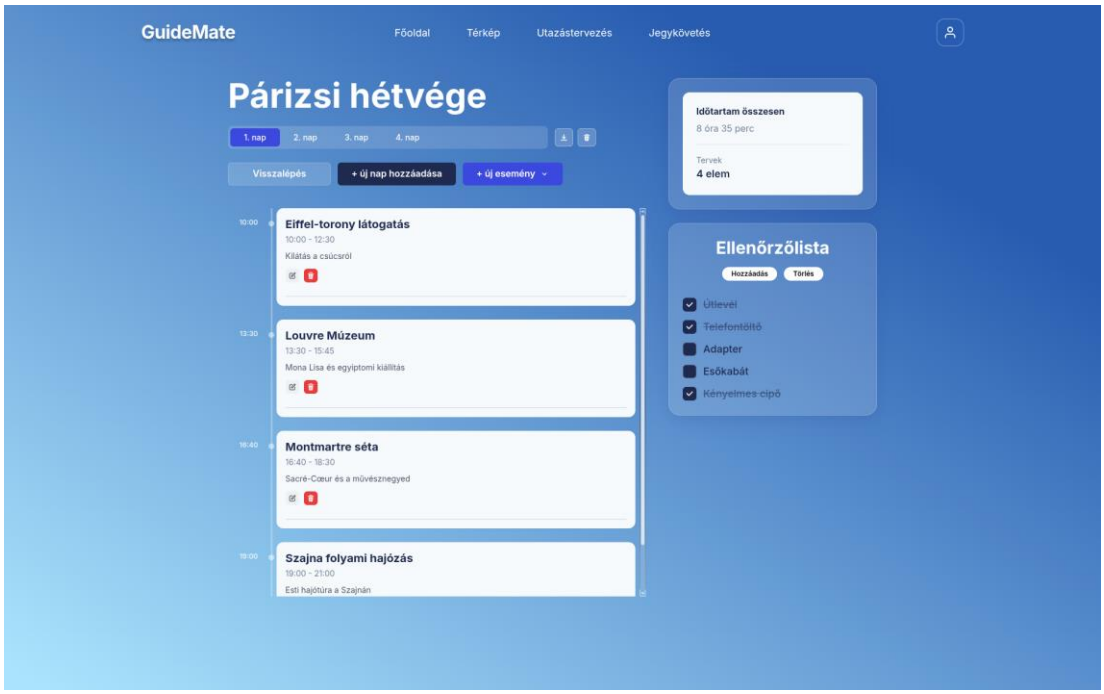
Az utazás törléséhez kattintson a törlés ikonra (kuka ikon) az utazás kártyáján. A rendszer megerősítést kér a törlés előtt. Fontos: a törlés visszavonhatatlan, az utazáshoz kapcsolódó összes program, ellenőrző lista és résztvevői rekord is törlődik.



12. ábra: Utazás törlésének megerősítő ablaka

5.2.4 Napi programok kezelése

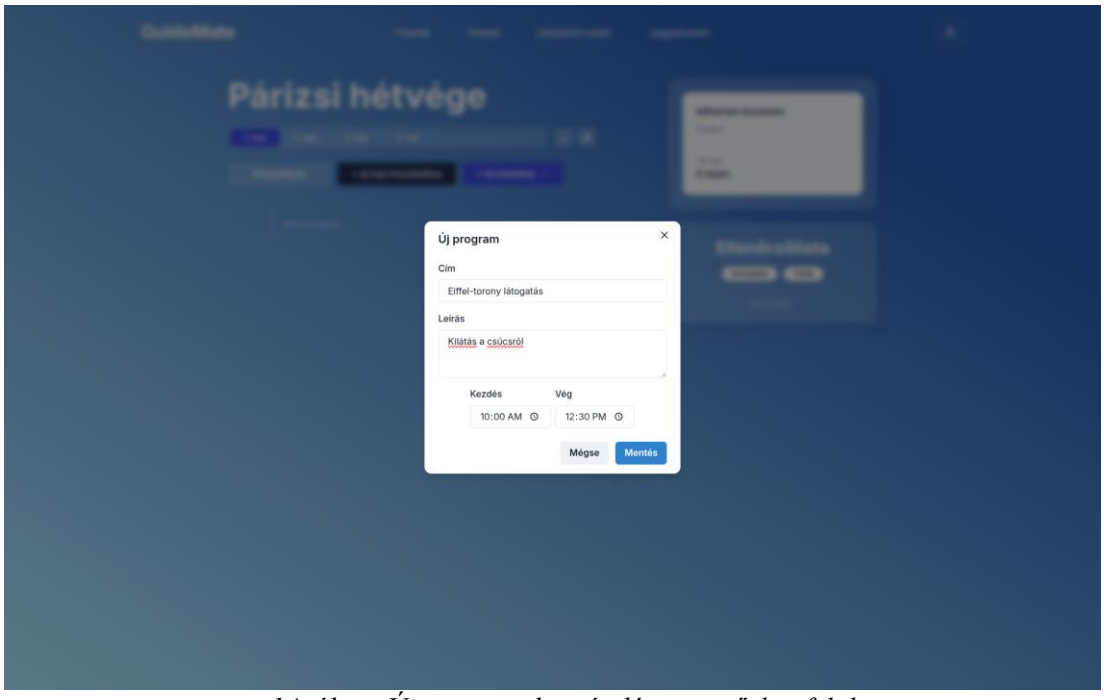
Az utazás nevére kattintva megnyílik az utazás részletei oldal, ahol a napi programok idővonalban jelennek meg.



13. ábra: Az utazás részletek oldala a napi programok idővonalával

Program hozzáadása

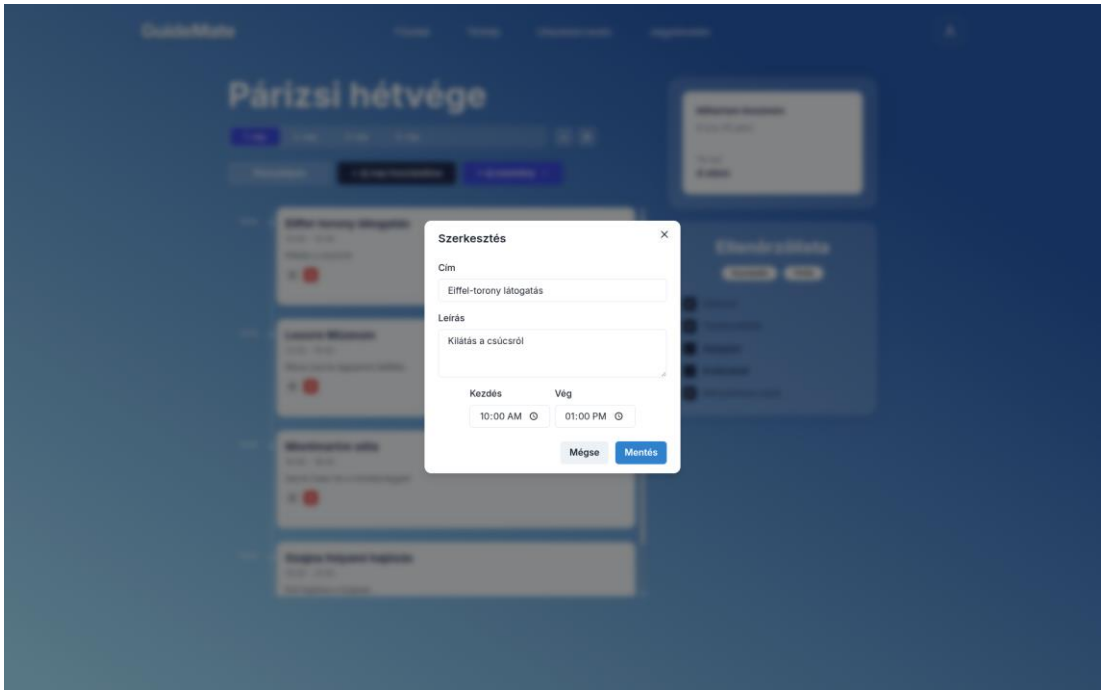
Kattintson az „Új program” gombra. A megjelenő űrlapon adja meg a program nevét, opcionális leírását, a dátumot, valamint a kezdő és záró időpontot (óra:perc formátumban). A Mentés gomb megnyomása után a program megjelenik az adott nap idővonalában. Ha a program dátuma az utazás időkeretén kívül esik, a rendszer automatikusan kiterjeszti az utazás dátumtartományát.



14. ábra: Új program hozzáadása – az űrlap felülete

Program szerkesztése

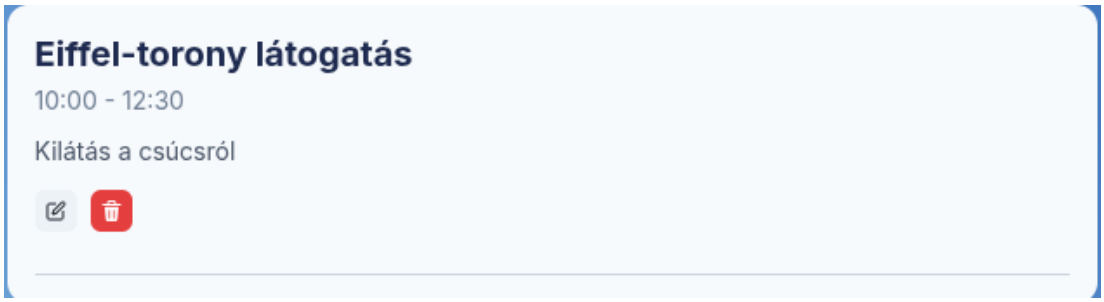
A program kártyáján kattintson a szerkesztés ikonra (ceruza ikon). Módosíthatja a program nevét, leírását, dátumát és időpontját. A változtatásokat a Mentés gomb véglegesíti.



15. ábra: Program szerkesztése – a szerkesztő űrlap

Program törlése

A program kártyáján kattintson a törlés ikonra (kuka ikon). A rendszer megerősítést kér a törlés előtt.



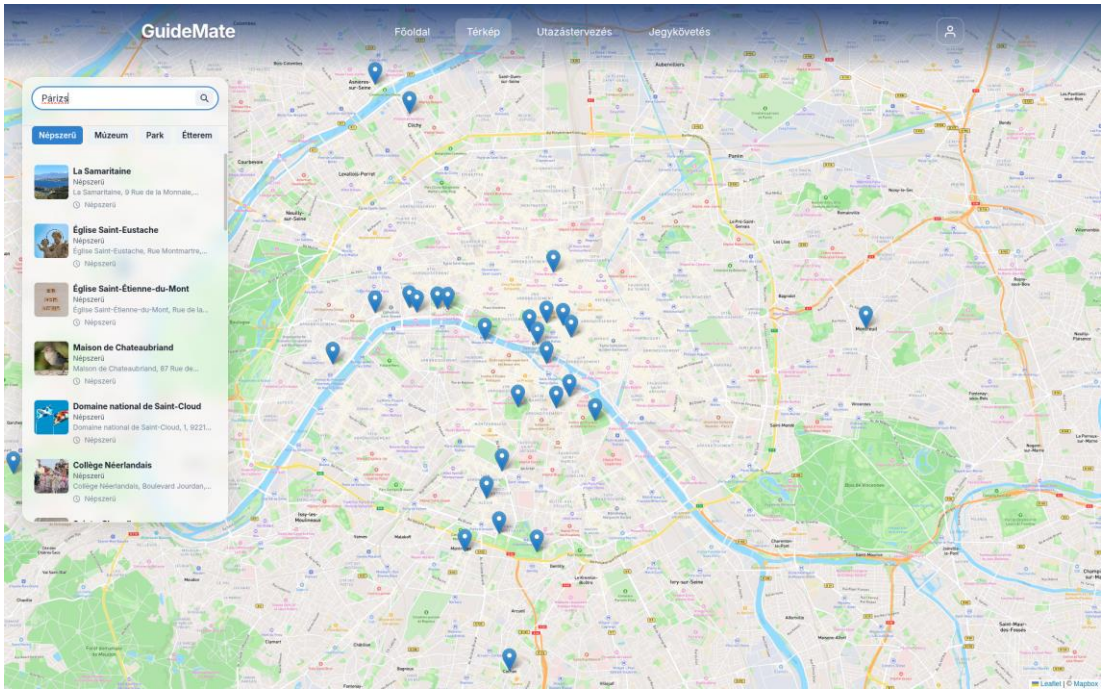
16. ábra: Program törlése

5.2.5 Interaktív térkép

A Térkép oldalon (menüpont) helyszíneket és látnivalókat kereshet.

Helyszín keresése

A keresőmezőbe gépelje be a kívánt helyszínt (például „Budapest”, „Párizs”), majd nyomjon Entert. A térkép a megtalált helyszínre ugrik, és a környező látnivalók automatikusan megjelennek jelölőkként.



17. ábra: Az interaktív térképoldal látnivalókereséssel

Látnivalók szűrése

A megjelenő látnivalókat kategória szerint szűrheti: Népszerű helyek, Múzeumok, Parkok, Éttermek. A szűrő gombok a térkép felett vagy mellett helyezkednek el.

18. ábra: Látnivalók szűrése kategória szerint

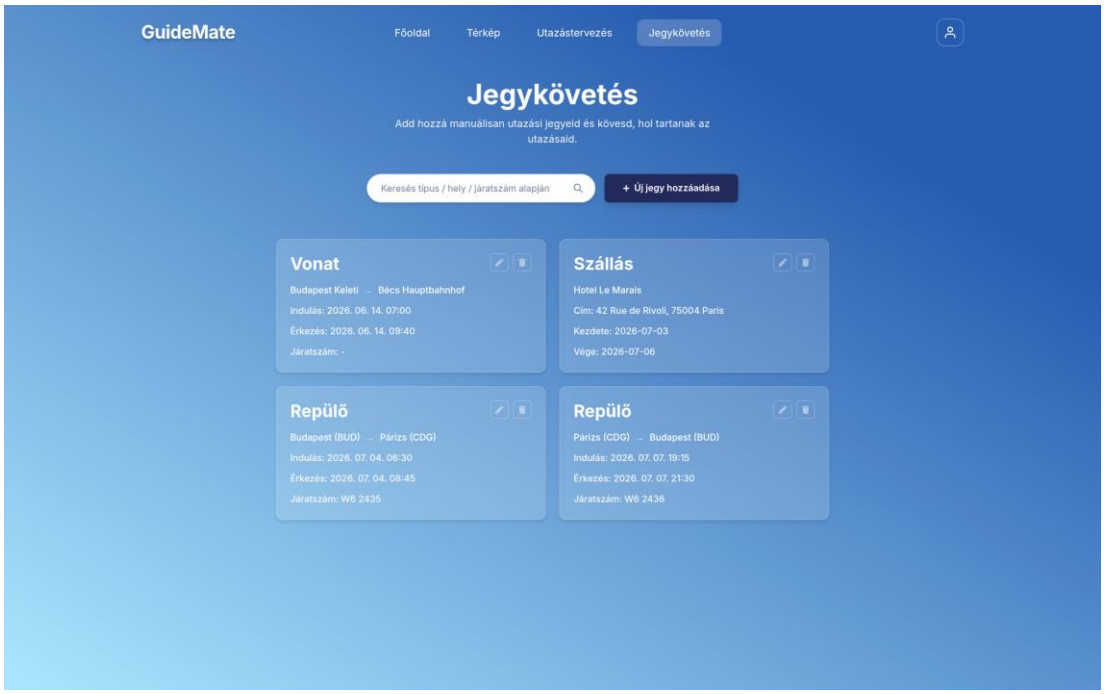
Látnivaló hozzáadása programhoz

Az egyes jelölőkre kattintva megjelenik a látnivaló neve, kategóriája és rövid leírása. A találatot hozzáadhatja az utazása valamelyik programjához a megfelelő gombra kattintva.

19. ábra: Találatok hozzáadása az utazáshoz

5.2.6 Jegykövetés (foglalások kezelése)

A Jegykövetés oldalon (menüpont) rögzítheti és áttekintheti összes foglalását.



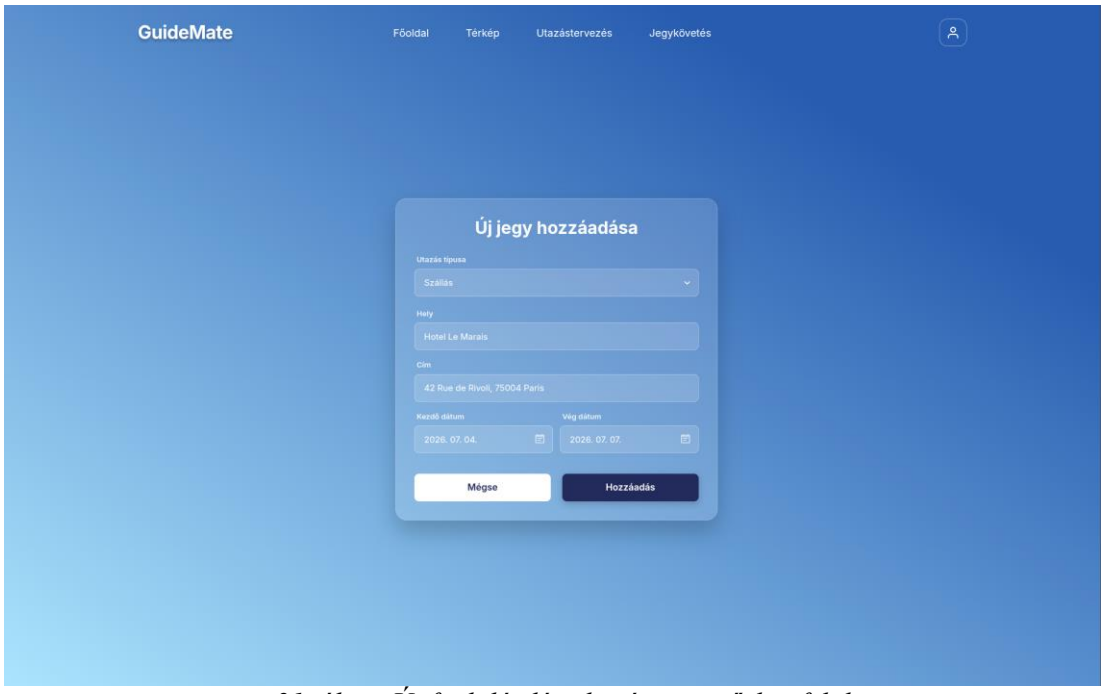
20. ábra: A jegykövetési oldal foglalási kártyákkal

Új foglalás létrehozása

Kattintson az „Új jegy” gombra. Válassza ki a foglalás típusát az alábbiak közül:

- Repülő: járatszám, légitársaság, indulási hely, érkezési hely, indulási és érkezési időpont.
- Busz: buszjárat száma, indulási hely, érkezési hely, indulási időpont.
- Vonat: vonatszám, indulási hely, érkezési hely, indulási időpont.
- Autó: bérlő cég, átvétel és visszaadás helye és időpontja.
- Szállás: szállás neve, címe, bejelentkezés és kijelentkezés időpontja.

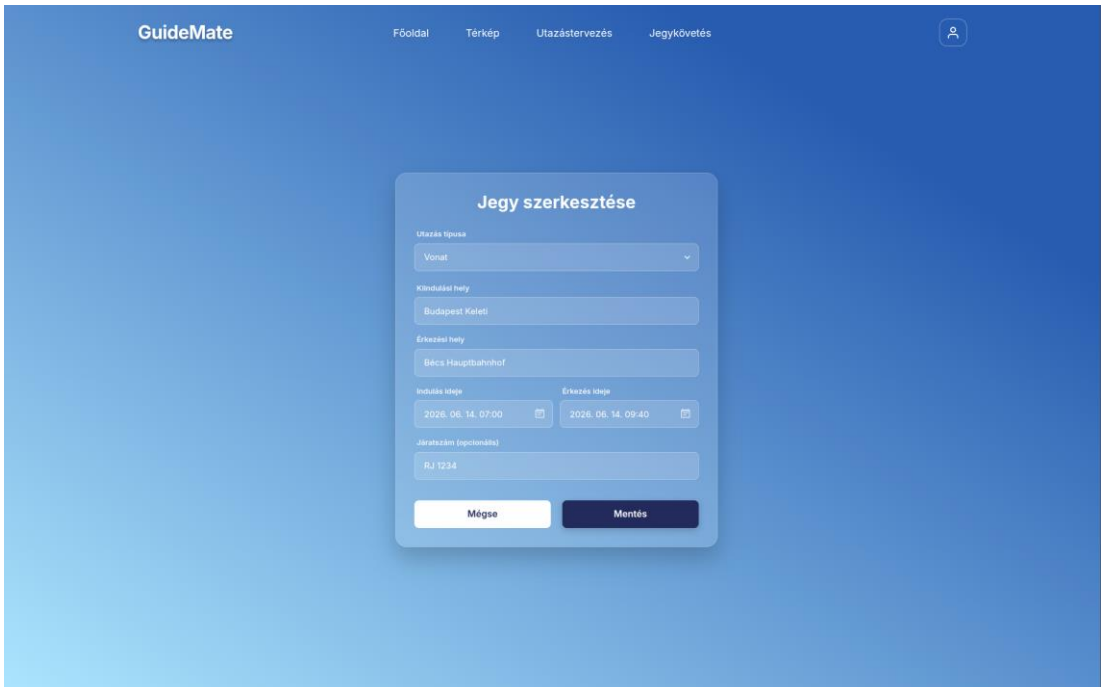
A típus kiválasztása után megjelennek a megfelelő beviteli mezők. Töltse ki az adatokat, majd kattintson a Mentés gombra.



21. ábra: Új foglalás létrehozása – az űrlap felülete

Foglalás szerkesztése

A foglalási kártyán kattintson a szerkesztés ikonra (ceruza ikon). Módosítsa a kívánt mezőket, majd kattintson a Mentés gombra a változtatások véglegesítéséhez.



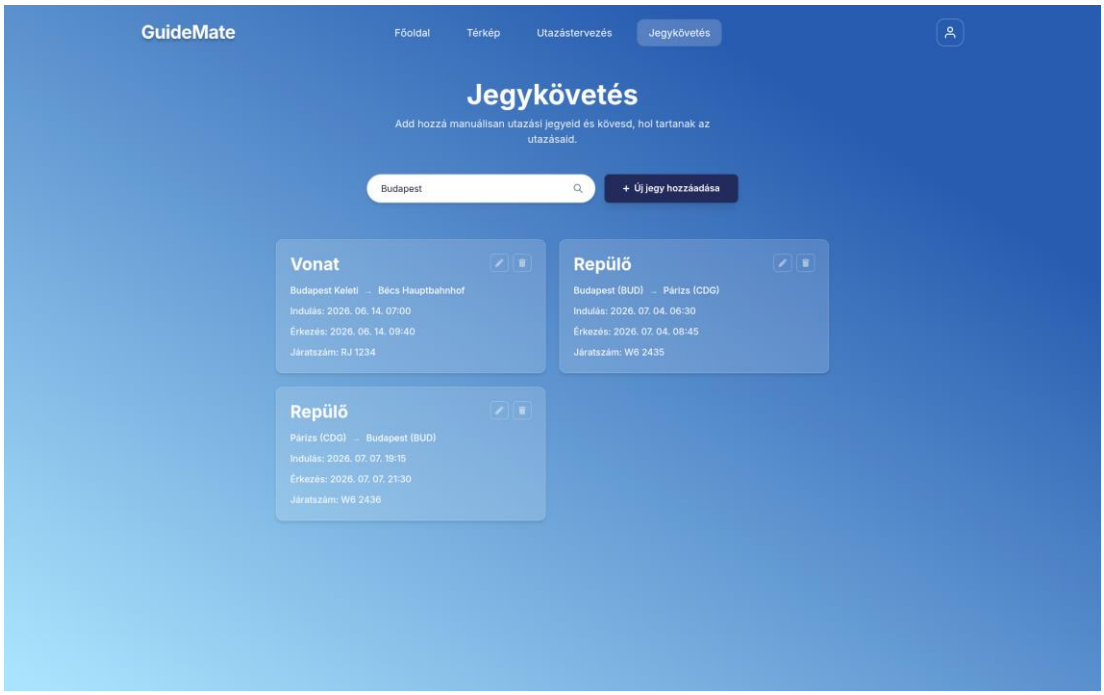
22. ábra: Foglalás szerkesztése – a szerkesztő űrlap

Foglalás törlése

A foglalási kártyán kattintson a törlés ikonra (kuka ikon). A rendszer megerősítést kér a törlés előtt.

Foglalások keresése

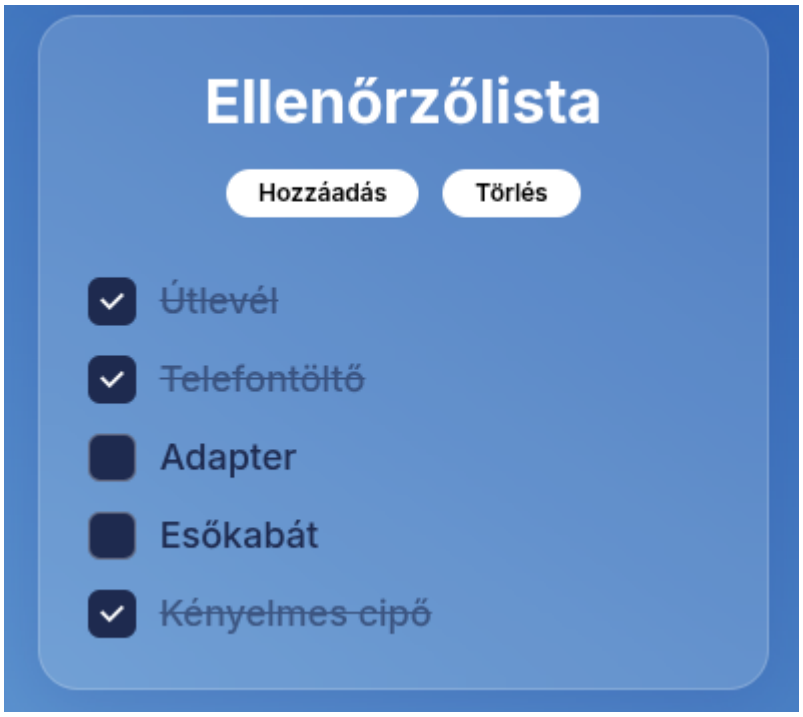
A keresőmezőbe gépelve szűrheti a foglalásokat típus, helyszín, járatszám vagy egyéb adatok alapján. A szűrés valós időben történik, az eredmények azonnal frissülnek.



23. ábra: Foglalások szűrése keresőkifejezéssel

5.2.7 Ellenőrző listák

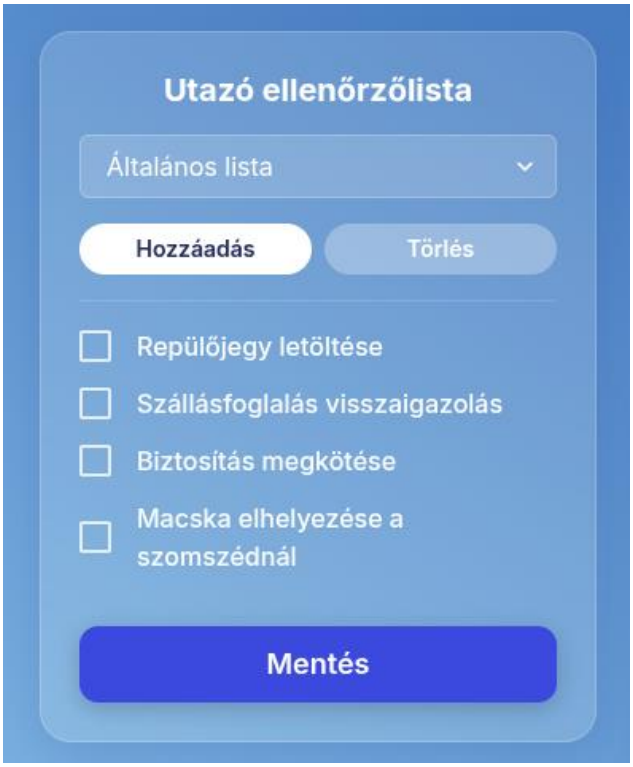
Az ellenőrző listák az Utazástervező oldal bal oldali paneljén érhetők el. Segítségükkel nyomon követheti, hogy minden fontos teendőt elintézett-e az utazás előtt.



24. ábra: Az ellenőrző lista panel

Utazás kiválasztása

A panel tetején egy legördülő menü található, amelyből kiválaszthatja, melyik utazásához szeretné kezelni az ellenőrzőlistát. Alapértelmezetten az "Általános lista" van kiválasztva, amely nem kapcsolódik egyetlen konkrét utazáshoz sem.



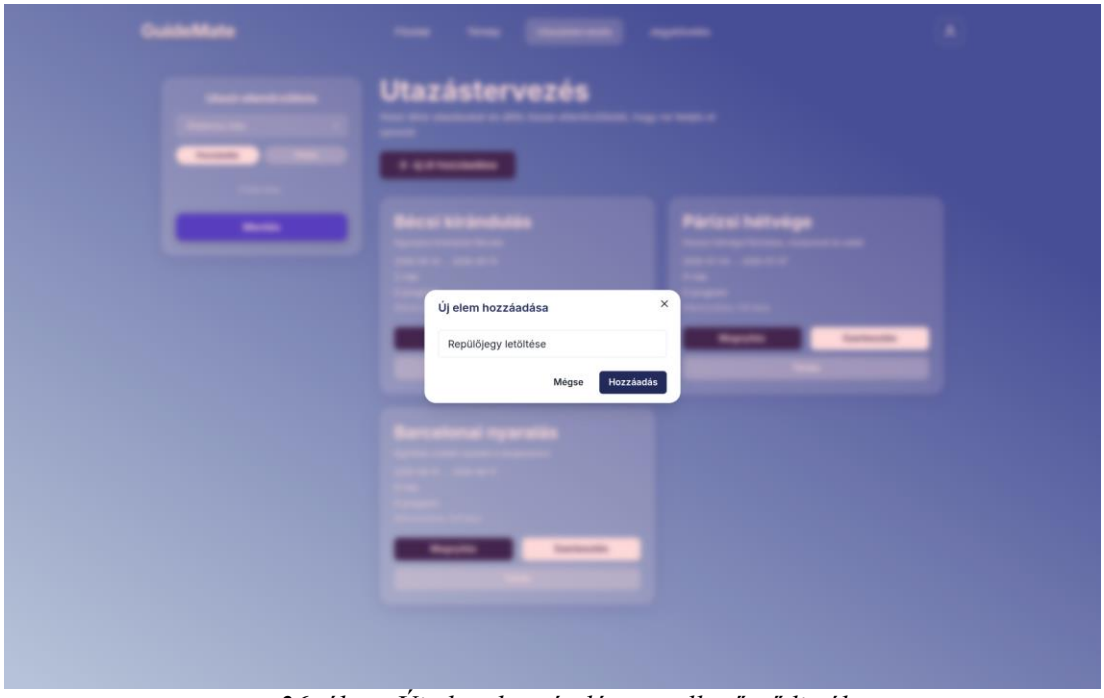
25. ábra: Az ellenőrző lista panel "Általános lista"-ja

Új lista létrehozása

A panel tetején található beviteli mezőbe gépelje be az új lista nevét, majd kattintson a „+” gombra. Az új lista megjelenik a panel alatt.

Elem hozzáadása

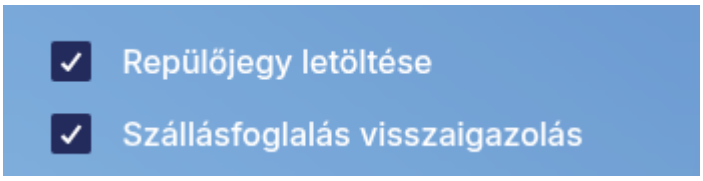
Kattintson a Hozzáadás gombra. A megjelenő ablakba írja be az új elem nevét, majd kattintson a megerősítés gombra vagy nyomjon Entert. Az elem megjelenik a listában, jelölőnégyzettel együtt.



26. ábra: Új elem hozzáadása az ellenőrző listához

Elem kipipálása

Az elem melletti jelölőnégyzetre kattintva megjelölheti az elemet készként. Az állapot azonnal mentődik a szerverre.



27. ábra: Kipipált elemek az ellenőrző listában

Elem törlése

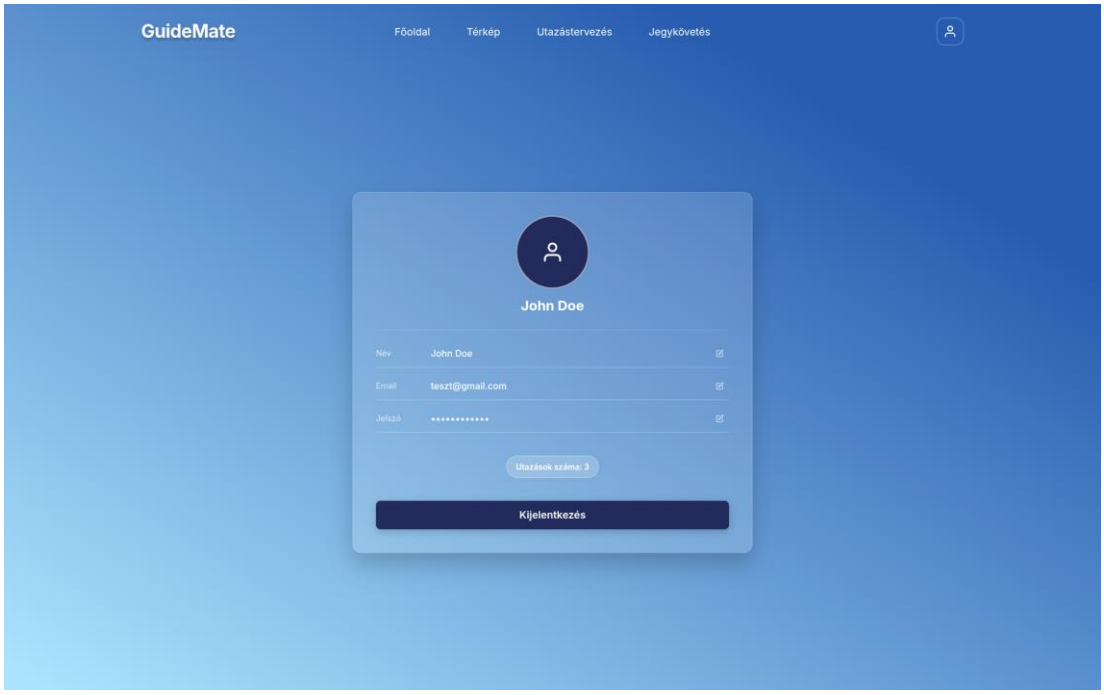
Pipálja ki az eltávolítani kívánt elemeket, majd kattintson a Törlés gombra. A gomb az összes kipipált elemet egyszerre távolítja el a listából.

Lista törlése

A lista neve mellett a törlés ikonra kattintva törölheti a teljes listát az összes elemével együtt. A rendszer megerősítést kér a törlés előtt.

5.2.8 Profilkezelés

A felső navigációs sávban a felhasználó nevére kattintva érhető el a Profil oldal.



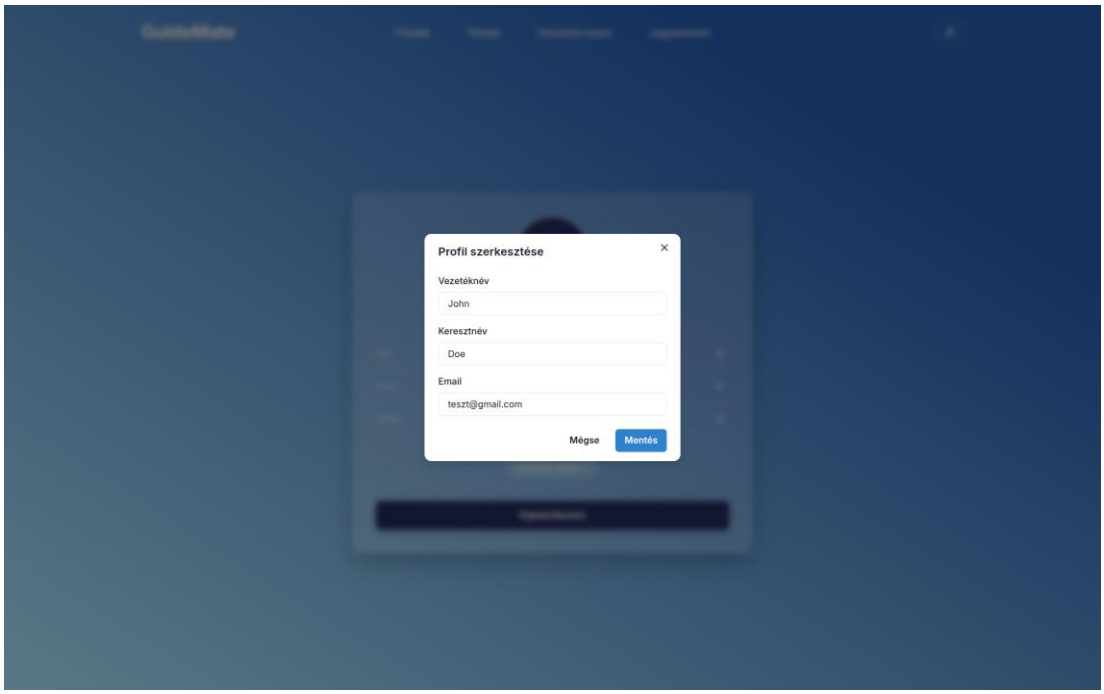
28. ábra: A profil oldal felülete

Profil megtekintése

A profil oldalon megtekintheti a regisztrációs adatait: vezetéknevét, keresztnevét, e-mail címét, a regisztráció dátumát és a szerepkörét (felhasználó vagy adminisztrátor).

Név módosítása

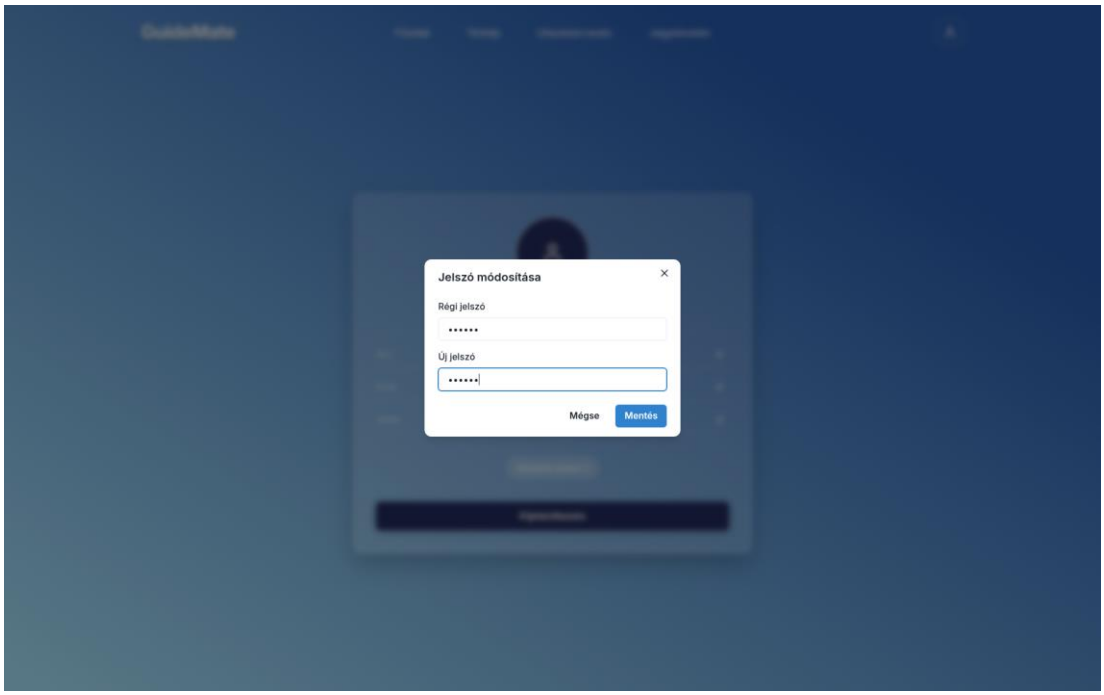
Kattintson a Szerkesztés gombra. Módosítsa a vezetéknevét és/vagy keresztnevét, majd kattintson a Mentés gombra a változtatások rögzítéséhez.



29. ábra: Profil szerkesztése – név módosítása

Jelszó módosítása

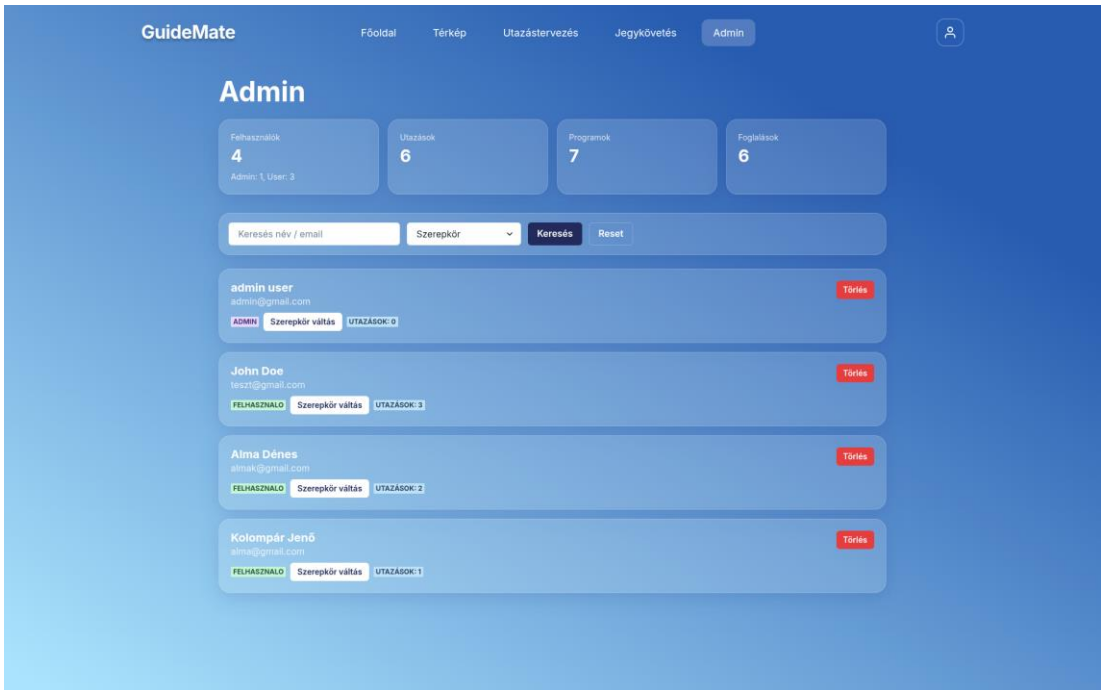
Kattintson a jelszó módosítása lehetőségre. Adja meg a jelenlegi jelszavát, majd az új jelszót kétszer (megerősítéssel). A Mentés gombra kattintva a jelszó frissül. Ha a jelenlegi jelszó nem egyezik, vagy az új jelszavak különböznek, hibaüzenet jelenik meg.



30. ábra: Jelszó módosítása űrlap

5.2.9 Adminisztrációs felület

Az adminisztrátori jogosultsággal rendelkező felhasználók számára az Admin menüpont érhető el a navigációs sávban.



31. ábra: Az adminisztrációs oldal felhasználói lista és statisztikák nézettel

Statisztikák megtekintése

Az Admin oldal tetején látható a rendszer statisztikai összefoglalója: az összes felhasználó, utazás, program, foglalás és ellenőrző lista száma, valamint a mai napi regisztrációk és utazáslétrehozások száma.

Felhasználók keresése

A felhasználói lista felett található keresőmezőbe gépelve szűrheti a felhasználókat név vagy e-mail cím alapján. Az eredmények valós időben frissülnek.

Szerepkör módosítása

A felhasználók listájában bármely felhasználó szerepköre módosítható a Szerepkör gombra kattintva. A módosítás azonnal érvénybe lép: a felhasználó szerepköre Felhasználó és Admin között váltható.

Felhasználó törlése

A felhasználó sorában a törlés ikonra (kuka ikon) kattintva törölheti a felhasználói fiókot. A rendszer megerősítést kér a törlés előtt. A törlés a felhasználó összes kapcsolódó adatát (foglalásait, résztvevői rekordjait) is eltávolítja.

32. ábra: Felhasználó törlésének megerősítő ablaka

6. Összefoglalás

A projekt során sikeresen megvalósítottuk a tervezett funkciókat: az utazások és napi programok kezelését, a foglaláskövető rendszert, az interaktív térképes helyszín- és látnivalókeresést, az ellenőrző lista rendszert, a JWT-alapú hitelesítést, a szerepkör alapú hozzáférés-szabályozást és az adminisztrációs felületet statisztikákkal.

A fejlesztés során számos technikai kihívással szembesültünk és néhány dolog változott. Bár néhány funkció működése eltér az eredeti elképzeléstől, a rendszer összességében megfelel a kitűzött céloknak. A látványtervhez képest is kellett változtatni a gördülékeny és egyszerű használat érdekében.

A legtöbb kihívást a térkép megvalósítása és az öt API (Nominatim, Geoapify, REST Countries, Pexels, Mapbox) összekötése okozta, amelyekből rengeteget tanultunk. Emellett a látványtervhez hű megjelenés elérése is időigényes feladat volt, mivel a végső felületet többször finomhangoltuk a könnyű kezelhetőség érdekében.

7. Kitekintés

A GuideMate jelenlegi állapotában egy stabil, működőképes alaprendszer. A jövőbeli fejlesztések több irányban továbbfejleszthető, amelyek az alkalmazást közelebb vinnék egy valódi, piaci termékhez.

7.1 Lehetséges továbbfejlesztési irányok

7.1.1 Kollaboratív funkciók

A jelenlegi rendszerben az utazások résztvevőkkel megoszthatók, de valódi együttes szerkesztés (például Google Docs mintájára) nem érhető el. WebSocket alapú valós idejű szinkronizáció lehetővé tenné, hogy több résztvevő egyszerre szerkeszthesse az utazás programját, és a változások azonnal megjelenjenek mindenkinél.

7.1.2 Mobilalkalmazás

A jelenlegi webalkalmazás reszponzív, de egy dedikált mobilalkalmazás (például React Native alapon) jobb felhasználói élményt nyújtana okostelefonon, különösen offline üzemmóddal és push értesítésekkel kiegészítve. Az offline mód lehetővé tenné az utazási adatok böngészését internetkapcsolat nélkül is.

7.1.3 AI-alapú ajánlórendszer

A jelenlegi látnivalókereső kategórián és koordinátán alapul. Egy mesterséges intelligencia alapú ajánlórendszer a felhasználó korábbi útjain, érdeklődési körein és értékelésein alapulva személyre szabott útvonalajánlatokat és programterveket készíthetne.

7.1.4 Időjárás-integráció

Az utazás dátumához és helyszíneihez kapcsolódó időjárás-előrejelzés megjelenítése a napi program nézet mellett hasznos kiegészítő információ lenne

7.1.5 Exportálás és megosztás

Az utazási terv PDF formátumban való exportálása, Google Naptárba való szinkronizálás, vagy az utazás nyilvános megosztási linkje hasznos funkciók lennének a közösségi élmény erősítéséhez.

7.1.6 Valutaváltó és költségkövetés

Egy valutaváltó és utazási költségkövetés modul segítené a felhasználókat az utazási büdzsé tervezésében és nyomon követésében. Ez kiegészülhetne diagramos megjelenítéssel a kategóriánkénti kiadásokról.

Irodalomjegyzék

TypeScript – https://www.typescriptlang.org/docs/

React – https://react.dev/

Vite – https://vitejs.dev/guide/

Chakra UI – https://chakra-ui.com/docs/getting-started

Leaflet – https://leafletjs.com/reference.html

NestJS – https://docs.nestjs.com/

Prisma ORM – https://www.prisma.io/docs

JSON Web Tokens (JWT) – https://jwt.io/introduction

Jest – https://jestjs.io/docs/getting-started

Vitest – https://vitest.dev/guide/

Nominatim (OpenStreetMap) – https://nominatim.org/release-docs/latest/

Geoapify – https://apidocs.geoapify.com/

REST Countries – https://restcountries.com/

Pexels API – https://www.pexels.com/api/documentation/

Ábrajegyzék

1. ábra: UML osztálydiagram az entitások mezőivel és metódusaival

2. ábra: Az adatbázis fizikai diagramja (ER diagram)

3. ábra: *Az adatbázis egyed-relációs diagramja*

4. ábra: Use-case diagram a rendszer főbb használati eseteiről

5. ábra: A TerkepOldal felülete helyszínkereséssel és Leaflet jelölőkkel

6. ábra: Az utazástervező főoldalának felülete

7. ábra: A regisztrációs oldal felülete

8. ábra: A bejelentkezési oldal felülete

9. ábra: Az utazástervező oldal utazáskártyákkal

10. ábra: Új utazás létrehozása – az űrlap felülete

11. ábra: Utazás szerkesztése – a szerkesztő felület

12. ábra: Utazás törlésének megerősítő ablaka

13. ábra: Az utazás részletek oldala a napi programok idővonalával

14. ábra: Új program hozzáadása – az űrlap felülete

15. ábra: Program szerkesztése – a szerkesztő űrlap

16. ábra: Program törlésének megerősítő ablaka

17. ábra: Az interaktív térképoldal látnivalókereséssel

18. ábra: Látnivalók szűrése kategória szerint

19. ábra: Találatok hozzáadása az utazáshoz

20. ábra: A jegykövetési oldal foglalási kártyákkal

21. ábra: Új foglalás létrehozása – az űrlap felülete

22. ábra: Foglalás szerkesztése – a szerkesztő űrlap

23. ábra: Foglalások szűrése keresőkifejezéssel

24. ábra: Az ellenőrző lista panel

25. ábra: Az ellenőrző lista panel "Általános lista"-ja

26. ábra: Új elem hozzáadása az ellenőrző listához

27. ábra: Kipipált elemek az ellenőrző listában

28. ábra: A profil oldal felülete

29. ábra: Profil szerkesztése – név módosítása

30. ábra: Jelszó módosítása űrlap

31. ábra: Az adminisztrációs oldal felhasználói lista és statisztikák nézettel

32. ábra: Felhasználó törlésének megerősítő ablaka

Táblázatjegyzék

1. táblázat: Az adatbázis entitásainak összefoglalása

2. táblázat: A backend főbb API-végpontcsoportjai

3. táblázat: Backend unit tesztek összefoglalása

4. táblázat: Frontend komponenstesztek összefoglalása

5. táblázat: Funkcionális tesztesetek

6. táblázat: Blackbox tesztesetek